

- 2 Using Spatial Analysis to Assess Spatiotemporal Trends in Overall Mortality
- 3 Project future mortality
- 4 Improve Predictions with Machine Learning
- 5 Zooming in on Mortality Causes

Project 2: Public Health & Epidemiology

Merrimack College DSE6630: Healthcare & Life Sciences Analytics

Katherine S. Geist, PhD

1 June 2026

##This section was completed by Lucia # 1 Introduction

It is **September of 2020**, right in the middle of the COVID-19 pandemic and during the California statewide shutdown. You have just been hired by the California Department of Public Health (CDPH) in Vital Records (<https://www.cdph.ca.gov/Programs/CHSI/Pages/Vital-Records.aspx>) to track non-COVID mortality trends for the past 4 years for which all data have currently been aggregated and de-identified (anonymized). You are told that the purpose is two-fold:

1. Help the state identify shifts in spatiotemporal mortality trends as a result of the COVID statewide stay-at-home orders.
2. Help the state build predictive models for non-COVID causes of mortality to establish a pre-COVID baseline.

1.1 Provided data: California mortalities

You were given properly de-identified data from 2017 - 2020, which contain mortality records for the state of California by gender, leading cause of death, race / ethnicity, and location of death. All data have been taken from the death certificates filed with the state.

NOTE: If you are interested in these data, the source is the California Death Profiles dataset (<https://data.chhs.ca.gov/dataset/death-profiles-by-county>) maintained by California Health and Human Services.

```
## Load the provided data files
load("california_mortality_tables.RData")
```

You are provided 6 tables total, which have the following variables.

Table 1. Data dictionary of variables in the California mortality tables.

All Tables	
Year	Year of record
Month	Month of record
County	California county where death was recorded
Annual Mortality Table	
annualCount	Number of deaths for the sex specified
raceEthnicity	Race and/or Ethnicity of the deceased; Latinx is unfortunately treated as a racial category here rather than an ethnic group
Specialized Variables	
leadingCauseDeath	Leading cause of death per death certificate; See separate table for encoding
ageBins	Age groups where the age at death was known
ageDeaths	Number of deaths for the age bin specified
biologicalSex	Biological sex of decedent
sexDeaths	Number of deaths for the sex specified
placeDeathOccurred	Location of death, per death certificate
placeDeaths	Number of deaths matching specified criteria
raceDeaths	Number of deaths matching race / ethnic criteria

totalDeaths Number of deaths per month, year, & county

Table 2. Underlying cause of death category based on coding using the Tenth Revision of the International Classification of Diseases (ICD) codes

3-letter Code	Description	ICD-10 Codes
HTD	Diseases of the Heart	I00-I09, I11, I13, I20-I51
CAN	Malignant Neoplasms (Cancers)	C00-C97
STK	Cerebrovascular Disease (Stroke)	I60-I69
CLD	Chronic Lower Respiratory Disease (CLRD)	J40-J47
INJ	Unintentional Injuries	V01-X59, Y85-Y86
PNF	Pneumonia and Influenza	J09-J18
DIA	Diabetes Mellitus	E10-E14
ALZ	Alzheimer's Disease	G30
LIV	Chronic Liver Disease and Cirrhosis	K70, K73-K74
SUI	Intentional Self Harm (Suicide)	U03, X60-X84, Y87.0
HYP	Essential Hypertension and Hypertensive Renal Disease	I10, I12, I15
HOM	Homicide	U01-U02, X85-Y09, Y87.1
NEP	Nephritis, Nephrotic Syndrome and Nephrosis	N00-N07, N17-N19, N25-N27]
OTH	All Other Causes of Death	All codes not listed above

NOTE: Make sure to refer to these tables as you go!

2 Using Spatial Analysis to Assess Spatiotemporal Trends in Overall Mortality

2.1 Background

Unlike your first project, your data engineering team (i.e., **me**) has provided you with most of the data tables in a nice, tidy format that are relatively nice and tidy to begin with - which means you can dig right into the analysis! Remember, any time you have datasets with **spatial information** they are spatial data; these data are organized by California county, so you know you must do a spatial analysis! However, there is also a *temporal* element - you have data from four different years, 2017 to 2020. So, you decide you will make maps to analyze both the spatial (geographic) and temporal (time) trends in the hope it will shed insight into which downstream analysis you should do. Consider this to be part of your **Exploratory Data Analysis**.

As discussed in lecture, one of the key types of spatial data files you might use are called **shapefiles**, which is what you are going to practice loading and working with shapefiles, using this open source dataset (https://gis.data.ca.gov/datasets/2c4e32ee61824e28bd4cf37ecac96f39_0/explore?location=37.271891%2C-119.270269%2C5) for the state of California. You will have to download these data as they are too large to put onto GitHub. **NOTE:** Make sure you download the shapefiles only and decompress them!

Begin by downloading the requisite spatial data files. Make sure you put them in their own directory called **California_County_Boundaries2025** on your Desktop; otherwise, you will need to adjust your path in the file settings below! This is similar to what I had you do in Demo 1, if you'll recall.

Notice that the directory contains six files, three of them with file extensions you discussed last week in lecture:

- California_County_Boundary.cpg
- California_County_Boundary.dbf
- California_County_Boundary.prj
- California_County_Boundary.shp
- California_County_Boundary.shp.xml
- California_County_Boundary.shx

Question 1 [1 point]:

What do each of these six files represent? You will not be able to open them directly on your computer, so you may need to refer back to lecture and do a little digging to figure out the type of data you would expect each of these files to contain.

A shapefile is not a single file but a collection of files that work together to store and manage geographic information. Each file serves a different purpose.

.shp: Contains the actual geographic features and geometry data. In this project, it stores the county boundary shapes used to draw the California map. .dbf: Stores attribute information associated with each geographic feature. Such as county names, identifiers, and other county-level variables. .shx: Serves as an index file for the shapefile. It allows GIS software to quickly locate and retrieve geographic features stored in the .shp file. .prj: Contains projection and coordinate reference system information. This ensures that the county boundaries are positioned correctly on the Earth and align properly with other spatial datasets. .cpg: Stores character encoding information used for text fields in the dataset. This helps ensure that county names and other text data are displayed correctly. .shp.xml: Contains metadata describing the dataset, such as information about its source, creation, structure, and intended use.

Together, these files allow geographic data to be accurately stored, displayed, and analyzed within GIS and spatial analysis software.

2.2 Your objective

You want to first get a sense for **annual trends** in overall mortality across the state to see what you are working with. Your first objective is to join your overall mortality data to the shapefile to be able to make maps to visualize these trends. You also want to calculate **spatial weights** to allow you to create predictive models of mortality by county.

2.3 Making your first map of California

You are going to practice two ways to make maps from shapefiles so that you can see two ways to do it. Also, to give you more for reference, you are roughly following the GIS Basics Chapter of the Epi R Handbook (https://epirhandbook.com/en/new_pages/gis.html) (with your own twist, of course!). This chapter may be helpful for understanding more of the “nuts and bolts” of GIS as well, including definitions and why you might do things certain ways.

The initial processing steps will include:

1. Import the shapefile
2. Filter rows to keep only areas of interest
3. Merge with the other data tables to create maps and analyze trends

2.3.1 Read in the shapefile & validate geometries

You will leverage the `read_sf()` function that is part of the `sf` package. As you might have guessed, `sf` refers to **shapefile**. Note that there is an alternative method to read in shapefiles called `st_read`, which is an alias for `read_sf()` with some different default arguments. By default, `read_sf()` is quiet, whereas `read_st()` will print information about the data source by default. Additionally, `read_sf()` returns a tibble rather than a dataframe. So which to choose? It's your preference, in this case. By default, your code below will use `read_sf()` but you could instead use `read_st()` if you wanted.

```
cali_sf <- read_sf("~/Desktop/Merrimack_DSE6630/Team_Directories/alpha/Module_Projects/Project_2/california_County_Boundaries2025")
```

```
## Alternatively, you can read in the shapefile using the 'st_read()' function
## cali_st <- st_read("~/Downloads/California_County_Boundaries2025/California_County_Boundary.shp")
```

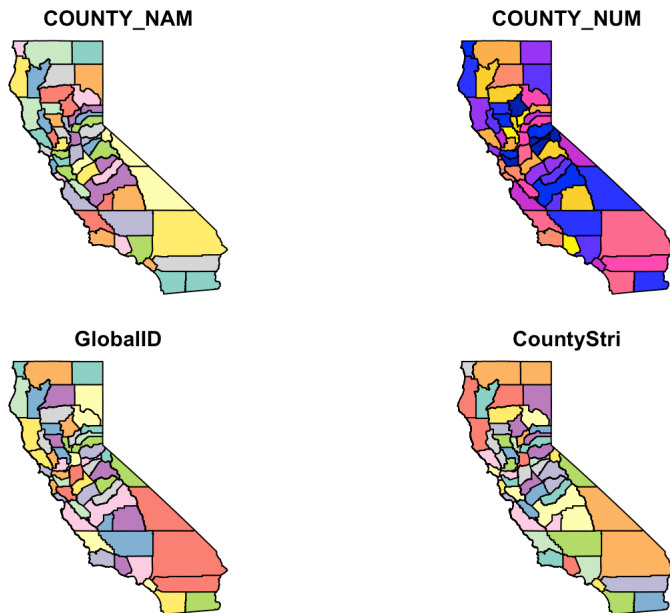
NOTE If you get a warning (as opposed to a critical error) is from GDAL (<https://gdal.org/en/stable/>), a library of raster and vector geospatial data formats that your `read_sf()` (or `st_read()`) functions are using to transform the polygons for mapping. This warning indicates that some polygon geometries in the shapefile have boundaries (rings) with an inconsistent order of coordinates (“invalid winding order”). GIS tools assume that it will read from the right-hand, and the warning is letting you know that it is autocorrecting it. However, you could still have issues and it isn't ideal.

Just as you did in Demo 2, you can fix this by validating the polygon boundaries:

```
## Disable Google S2 engine
sf_use_s2(FALSE)
## Validate the geometry boundaries/edges
cali_sf <- st_make_valid(cali_sf)
```

Now, let's do a quick check of the map of California counties to make sure all the layers look okay:

```
plot(cali_sf)
```



And let's check the validity of the boundaries:

```
st_is_valid(cali_sf)
```

```
## [1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [16] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [31] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [46] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
```

```
print(cali_sf$COUNTY_NAM)
```

```
## [1] "Alpine"      "Amador"      "Butte"       "Calaveras"
## [5] "Colusa"     "Contra Costa" "Del Norte"   "El Dorado"
## [9] "Fresno"     "Glenn"       "Humboldt"    "Imperial"
## [13] "Inyo"       "Kern"        "Kings"       "Lake"
## [17] "Lassen"     "Los Angeles" "Madera"      "Marin"
## [21] "Mariposa"   "Mendocino"   "Merced"      "Modoc"
## [25] "Mono"       "Monterey"    "Napa"        "Nevada"
## [29] "Orange"     "Placer"      "Plumas"      "Riverside"
## [33] "Sacramento" "San Benito"   "San Bernardino" "San Diego"
## [37] "San Francisco" "San Joaquin" "San Luis Obispo" "San Mateo"
## [41] "Santa Barbara" "Santa Clara" "Santa Cruz"    "Shasta"
## [45] "Sierra"      "Siskiyou"    "Solano"       "Sonoma"
## [49] "Stanislaus"  "Sutter"      "Tehama"       "Trinity"
## [53] "Tulare"     "Tuolumne"    "Ventura"      "Yolo"
## [57] "Yuba"       "Alameda"     "Colusa"
```

```
nrow(cali_sf)
```

```
## [1] 59
```

```
length(unique(cali_sf$COUNTY_NAM))
```

```
## [1] 58
```

Question 2 [1 point]:

Why did TRUE print 69 times? What are the "69 features" in the geometry?

Hint 1: Are all of them counties? Try running `print(cali_sf$COUNTY_NAME)`. What do you see?

The `st_is_valid()` function checks whether each geometry feature in the shapefile is valid and returns either TRUE or FALSE for each feature. In our shapefile, TRUE printed 59 times because there are 59 geometry features and all of them were valid after running `st_make_valid()`. The 59 features do not represent 59 unique counties. When we checked the county names, there were only 58 unique counties. This means that at least one county is represented by more than one geometry feature. In GIS data, a single county can sometimes be stored as multiple polygons if it contains disconnected geographic areas. Therefore, the number of geometry features can be larger than the number of unique counties.

Next, let's take a look at the geometry of the shapefile that you loaded.

```
cali_sf$geometry
```

```
## Geometry set for 59 features
## Geometry type: GEOMETRY
## Dimension:      XY
## Bounding box:   xmin: -13849230 ymin: 3833650 xmax: -12704990 ymax: 5162402
## Projected CRS: WGS 84 / Pseudo-Mercator
## First 5 geometries:
```

Notice that, just like Demo 2, you only have the **polygon** geometry in your shapefile. If you were to investigate further, you would find that currently there are no other geometries (e.g., lines or points) in this shapefile. You could add them and export the shapefile, if you ever needed to do so, like to share with someone else. For example, if you wanted to add points for the county seats of each county, you could add them based on longitude and latitude; similarly, you could add just points for major California cities, which I will demonstrate below.

Question 3A [0.5 points]:

What is the bounding box that comes with your shapefile and what does it tell us? **What would you do to the bounding box if you wanted to zoom in or out on a geographic feature?**

The bounding box is the smallest rectangular area that contains all of the geographic features in the shapefile. For this dataset, the bounding box is defined by `xmin = -13849230`, `ymin = 3833650`, `xmax = -12704990`, and `ymax = 5162402`. These values represent the minimum and maximum x- and y-coordinates of the California county boundaries and define the overall spatial extent of the map. The bounding box tells us how much geographic area is covered by the shapefile and where the features are located within the coordinate system. If we wanted to zoom in on a specific geographic feature, we would decrease the size of the bounding box by narrowing the coordinate ranges. If we wanted to zoom out, we would increase the coordinate ranges so that a larger geographic area is displayed.

Question 3B [0.5 points]:

What is the projection? What does it tell you and why does projection matter? **What will change if you change the projection?**

The projection for this shapefile is WGS 84 / Pseudo-Mercator. A projection is a mathematical method used to represent the Earth's curved surface on a flat map. The projection determines how geographic coordinates are translated into map coordinates and how spatial features are displayed. Projection matters because different projections can distort distance, area, shape, or direction. Spatial analyses such as calculating distances, areas, and spatial relationships depend on the coordinate system being used. If an inappropriate projection is selected, the results of the analysis may be inaccurate. If the projection is changed, the actual county boundaries do not change, but their coordinate values and appearance on the map may change. Different projections emphasize different geographic properties, such as preserving area, shape, or distance, which can affect how the data are visualized and analyzed.

2.3.2 Create a basic map of California

In Demo 2, I showed you how to plot shapefiles using `ggplot2`'s `geom_sf()` function, and even though there are advanced mapping functions in R you could use you will stick with that for consistency, convenience, and speed.

```
## Initiate the plot and tell it the dataset
ggplot(data = cali_sf) +
  ## Give it specific fill and border colors
  geom_sf(fill = skittles[6],
    color = "gray20",
    linewidth = 0.25,
    alpha = 0.65) +

  ## Set map projection explicitly
  coord_sf(expand = FALSE) + ## <- keeps map tight to its bounding box

  ## Uncomment to make a completely blank map background
  theme(
    panel.grid = element_blank(),
    panel.background = element_rect(fill = "white"),
    plot.background = element_rect(fill = "white"),
    axis.text = element_blank(),
    axis.ticks = element_blank(),
    axis.title = element_blank(),
  ) +

  ## Give it a title and footnote/caption
  labs(title = "California Counties",
    caption = "Source: California State Geoportal")
```

California Counties



Source: California State Geoportal

2.3.3 Zooming in on the map

Now, what if you only wanted to visualize a few counties? Let's look at the three northernmost counties that border Oregon. The three northernmost California counties that border Oregon are:

- Del Norte
- Siskiyou
- Modoc

Notice that the county names are accessible through `cali_sf$COUNTY_NAME`. Thus, you can just use the `filter()` function from `dplyr` to be able to grab the three northernmost counties you want by name, which I have started for you.

Question 4 [1 point]:

Finish the code below to create a map of the northernmost 3 counties in California. You can copy and paste, with minor modifications, the code for the entire state map to do this. **NOTE** that you are storing your map into `map1`. This is because I will show you how to add labels in the next section! Don't forget to update your plot title for full credit!

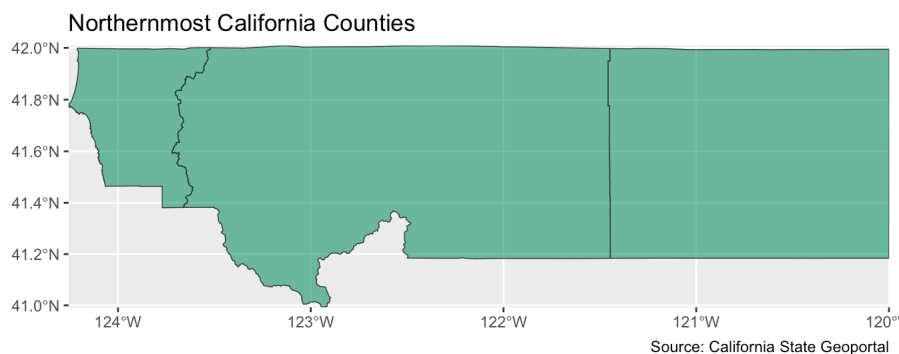
```

### Start from the California spatial tibble - Uncomment to run
map1 <- cali_sf %>%
# ## Filter to just the 3 counties that border Oregon - Uncomment to run
  filter(COUNTY_NAM == "Del Norte" | COUNTY_NAM == "Siskiyou" | COUNTY_NAM == "Modoc") %>%

## YOUR CODE HERE.
  ggplot() +
  geom_sf(
    fill = skittles[6],
    color = "gray20",
    linewidth = 0.25,
    alpha = 0.65
  ) +
  coord_sf(expand = FALSE) +
  labs(
    title = "Northernmost California Counties",
    caption = "Source: California State Geoportal"
  )

map1

```



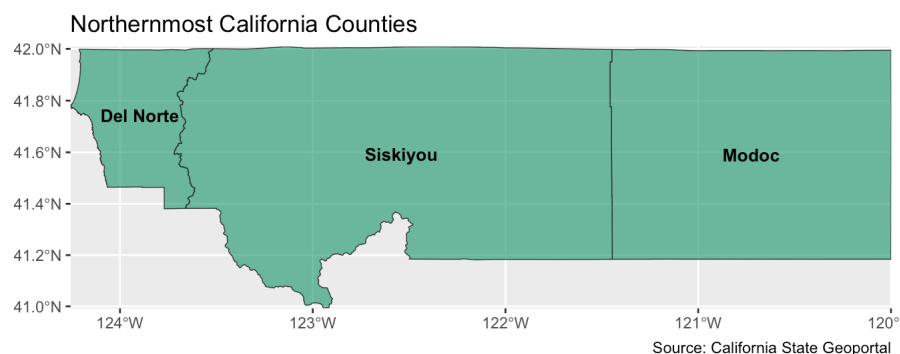
Now let's add labels! This code will not run unless you have made `map1` correctly!

```

### UNCOMMENT THIS CODE TO RUN:

# ## Calculate the county centers:
county_centroids <- st_centroid(cali_sf) %>%
# ## Filter for just the northern centroids
  filter(COUNTY_NAM == "Del Norte" | COUNTY_NAM == "Siskiyou" | COUNTY_NAM == "Modoc")
#
# ## Now, add the labels to the map:
map1 + geom_sf_text(data = county_centroids,
  aes(label = COUNTY_NAM),
  size = 3.5, fontface = "bold") +
  ## fix the x and y axis labels to make a beautiful map!
  theme(axis.title = element_blank())

```



2.3.4 Creating points for major cities in California

Often, your stakeholders will want geographic reference points; whether this is the label of a county name, as you just did for the 3 counties that border Oregon, or they might be locations of major cities. Let's choose to plot five cities in California, the capital Sacramento and the four most populated cities:

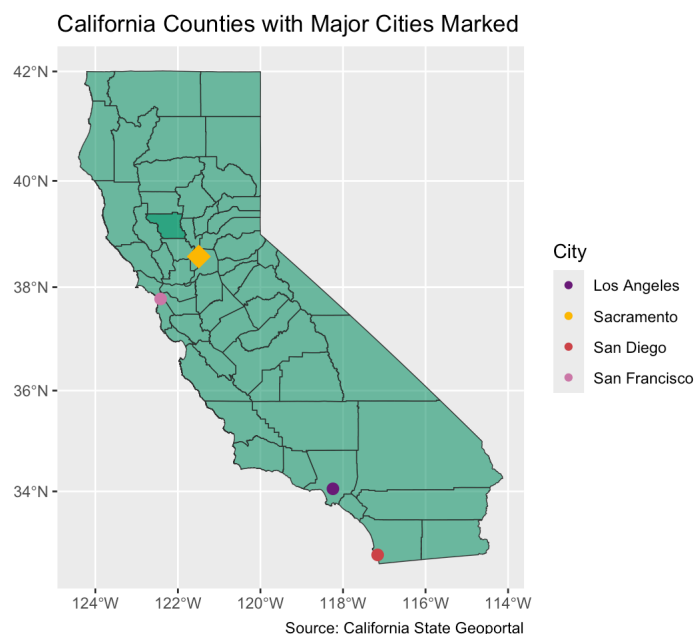
- Sacramento (the capital)
- Los Angeles
- San Diego
- San Francisco

But, to do this, it takes a little sorcery. You have to create a second **layer** for the points you want to overlay. So, you will put the longitude and latitude for each city, with the city's name and an identifier as to whether it's the capital or not, into its own dataframe. You will then convert it to a shapefile object.

```
## Create a cities data frame with coordinates
cities <- data.frame(
  city = c("Sacramento", "San Diego", "Los Angeles", "San Francisco"),
  lon = c(-121.4944, -117.1611, -118.2437, -122.4194),
  lat = c(38.5816, 32.7157, 34.0522, 37.7749),
  cityType = c("capital", "major", "major", "major")
)

## Convert to shapefile object with the same projection as what you had
cities_sf <- st_as_sf(cities, coords = c("lon", "lat"), crs = 4326)
```

Now, you can plot the map by overlaying the city points, with differing colors for each city and a different shape and size marker for the capital, Sacramento:



Question 5 [1 point]:

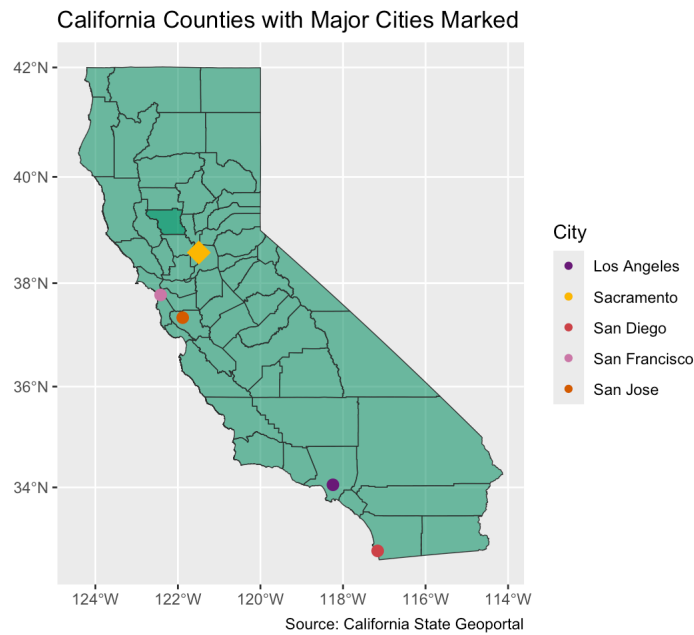
Your stakeholders point out that you have forgotten San Jose, one of the most populated cities in California! They ask you to go back to the map and add it on.

Hint 1: The coordinates of San Jose are 37.3387° N, 121.8853° W **Hint 2:** Don't forget to add a `cityType` for San Jose! **Hint 3:** Don't forget to add a skittles color for San Jose in `scale_color_manual`!

```
## Create a cities data frame with coordinates, now including San Jose
cities <- data.frame(
  city = c("Sacramento", "San Diego", "Los Angeles", "San Francisco", "San Jose"),
  lon = c(-121.4944, -117.1611, -118.2437, -122.4194, -121.8853),
  lat = c(38.5816, 32.7157, 34.0522, 37.7749, 37.3387),
  cityType = c("capital", "major", "major", "major", "major")
)

## Convert to shapefile object
cities_sf <- st_as_sf(cities, coords = c("lon", "lat"), crs = 4326)

## Plot California counties with city points, including San Jose
ggplot(data = cali_sf) +
  geom_sf(
    fill = skittles[6],
    color = "gray20",
    linewidth = 0.25,
    alpha = 0.65
  ) +
  coord_sf(expand = FALSE) +
  geom_sf(
    data = cities_sf,
    aes(color = city, shape = cityType, size = cityType)
  ) +
  scale_shape_manual(values = c(capital = 18, major = 16), guide = "none") +
  scale_size_manual(values = c(capital = 6, major = 3), guide = "none") +
  scale_color_manual(values = skittles[1:5], name = "City") +
  labs(
    title = "California Counties with Major Cities Marked",
    caption = "Source: California State Geoportal"
  )
)
```



2.4 Joining shapefiles with a tibble or dataframe

As mentioned above, the `sf_read()` function from the `sf` package loads your shapefile as a `tibble`, which makes life much easier for you when dealing with shapefiles. It also means you can easily join your shapefiles with other tibbles / dataframes, like the ones we've imported of mortality data. Let's briefly breakdown what your process to do this is going to look like:

Our Process:

1. Rename the **County** column so it matches the shapefile
2. Summarize the data by **Year** so you can visualize the annual trends
3. Join the two tibbles by `COUNTY_NAME`
4. Make sure to turn the joined tibble **back into a shapefile** using the `st_as_sf()` function, otherwise it will fail. The join will convert it into a structure that is no longer a shapefile, so you have to back-convert it to a shapefile!

Question 6 [1 point]:

Annotate the following code chunk with comments to make sure you understand what you are doing here. **HINT:** refer back to the earlier tables with the data descriptions to help you understand the data better!

```
## Start with the totalDeaths dataset because it contains the county-level
## mortality counts that we want to visualize on a map
mortality_sf <- totalDeaths %>%
  ## Rename the County column so that it matches the county name
  ## column in the California shapefile and can be joined correctly
  rename(COUNTY_NAME = County) %>%
  ## Group the mortality data by county, year, and month so that
  ## deaths can be summarized within each county-time period
  group_by(COUNTY_NAME, Year, Month) %>%
  ## Calculate the total number of deaths for each county-year-month
  ## combination while ignoring missing values
  summarise(totalDeaths = sum(totalDeaths, na.rm = TRUE)) %>%
  ## Join the mortality data to the California county shapefile using
  ## COUNTY_NAME as the matching variable. The relationship is set to
  ## "many-to-many" because multiple mortality records exist for each
  ## county (different months and years), and the shapefile may also
  ## contain multiple geometry features for a county.
  full_join(cali_sf, by = "COUNTY_NAME", relationship = "many-to-many") %>%
  ## Convert the joined dataset back into an sf object so it retains
  ## its spatial geometry and can be mapped and analyzed spatially
  st_as_sf()
```

Question 7A [0.5 points]:

What does `st_join()` do? Could you have executed the join with `st_join()` instead of `full_join()` here? Why or why not?

The `st_join()` function performs a spatial join by matching features based on their geographic locations and spatial relationships rather than a common column. For example, it can determine whether a point falls within a polygon or whether two geographic features overlap. In this case, `st_join()` would not be the most appropriate choice because the mortality data and the California shapefile share a common county name variable (`COUNTY_NAM`). The goal is to join the datasets using county names rather than spatial relationships. Therefore, `full_join()` is more appropriate because it matches records using the shared county identifier.

Question 7B [0.5 points]:

This question has three subparts.

1. How many features are there in `mortality_sf`?

```
nrow(mortality_sf)
```

```
## [1] 2596
```

2. Why are there so many more than in `cali_sf`? > There are 2,596 features in `mortality_sf`. There are many more features because the mortality dataset contains multiple observations for each county across different months and years. When the mortality data were joined to the shapefile, the county geometries were repeated for each county-month-year observation. As a result, the number of rows increased substantially from the original shapefile.
3. How many counties are there now in `COUNTY_NAM`? (Hint: Use `unique()` or `n_distinct()` to help you get the true number!) Why is it different from the number you saw in Question 2?

```
n_distinct(mortality_sf$COUNTY_NAM)
```

```
## [1] 58
```

There are 58 unique counties in `COUNTY_NAM`. This differs from the 59 geometry features observed in Question 2 because Question 2 counted spatial features, while this question counts unique county names. One county was represented by more than one geometry feature in the shapefile, which is why there were 59 features but only 58 unique counties. After the join, the counties appear many times due to repeated monthly and yearly observations, but the number of unique counties remains 58.

2.5 Visualizing annual trends in overall mortality

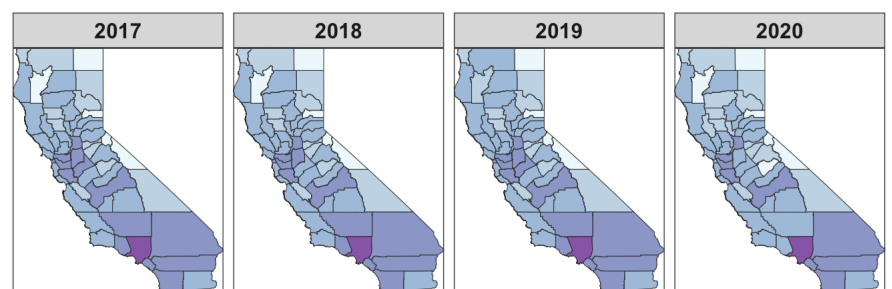
You are asked to make four maps, one for each year, that shows how annual overall mortality trends to identify any changes as a result of either the COVID-19 pandemic or the stay-at-home orders. Notice that you first use a `group_by()` and a `summarize()` to tally the total deaths per county per year - but **unlike your previous choropleths**, you are applying a purposeful set of cuts every 5×10^4 deaths. Why? This is to allow you to see order of magnitude (https://en.wikipedia.org/wiki/Order_of_magnitude) changes in the total deaths by California counties for each of the four years.

```
## Start from the mortality spatial tibble
mortalitySummary <- mortality_sf %>%
  ## Group by and then summarize to total the deaths by County and Year
  group_by(COUNTY_NAM, Year) %>%
  summarise(totalDeaths = sum(totalDeaths, na.rm = TRUE)) %>%
  mutate(deathLabels = cut(totalDeaths,
                           breaks = c(0, 50, 500, 5000, 50000, Inf),
                           labels = c("0-49", "50-499", "500-4,999",
                                       "5,000-49,999", "50,000+"),
                           ordered_result = TRUE, right = F))

## Initialize the plot
ggplot(mortalitySummary) +
  ## Chloropleth: draw the polygons of the counties and color by total deaths
  geom_sf(aes(fill = deathLabels), color = "gray20", size = 0.2) +
  ## Apply manual fill scale for total deaths
  scale_fill_manual(
    name = "Deaths, All Causes (per 10,000 persons)",
    values = c4a(palette="BuPu", n=6),
    drop = FALSE
  ) +
  ## Set map projection explicitly
  coord_sf(expand = FALSE) + ## <- keeps map tight to its bounding box
  theme_bw() +
  ## Make a completely blank map background this time
  theme(
    panel.grid = element_blank(),
    panel.background = element_rect(fill = "white"),
    plot.background = element_rect(fill = "white"),
    axis.text = element_blank(),
    axis.ticks = element_blank(),
    axis.title = element_blank(),
    strip.text = element_text(size = 12, face = "bold"),
    legend.position = "bottom"
  ) +
  labs(
    title = "California Annual Deaths by County (All Causes)",
    subtitle = "2017 - 2020",
    caption = "Source: California DPH Vital Records"
  ) +
  ## Use facet_wrap to make one map per year
  facet_wrap(~ Year, ncol = 4)
```

California Annual Deaths by County (All Causes)

2017 - 2020



Deaths, All Causes (per 10,000 persons) 0-49 50-499 500-4,999 5,000-49,999 50,000+

Source: California DPH Vital Records

2.5.1 Uh oh, can you spot the problem?

Question 8 [1 point]:

Is there any pattern that is a cause for concern in your interpretation of **total deaths**?

Hint 1: The county with the highest number of deaths (darkest purple) is **Los Angeles county**. What is special about LA county? **Hint 2:** Do you see other similar darker colors in counties around cities?

A major concern with interpreting the maps is that they display total deaths rather than a mortality rate adjusted for population size. Los Angeles County consistently appears as the darkest county, but this is likely because it has the largest population in California rather than because it has the highest mortality risk. A similar pattern can be seen in other counties that contain large cities or metropolitan areas, such as San Diego, Sacramento, Alameda, and Santa Clara counties. These counties tend to have higher numbers of deaths simply because more people live there. As a result, the maps may be reflecting differences in population size rather than differences in health outcomes or mortality risk. To make a more meaningful comparison across counties, it would be better to calculate a mortality rate (for example, deaths per 100,000 people) rather than using total deaths alone. ## 2.6 Accessing the 2020 US Census API

Hopefully, you have figured out that you need population data to add to your dataframes. In Demo 2, I gave you Census data - but this time, I want to teach you how to more efficiently and iteratively access data through an **API** (Application Programming Interface), which acts as a bridge between two software programs to exchange data. As data scientists, learning to use and love APIs will serve you well! *Thus, for the Census data this time you are going to access the Census Bureau's API endpoint.*

If you have not yet worked with an API, never fear! That's why we're doing it here. Here, the Census API will allow you to access their database and download their data in JSON (Java Script Object Notation) format, which is format that **flattens** databases into text. By connecting to the API endpoint for the Census Bureau, you will be able to download instantaneously only the data you want, which in your case will be population sizes and some basic demographics about the breakdown of population by sex and by individuals over the age of 65 years.

Because the US Census API uses JSON formatted files, you will be using the `jsonlite` package to convert the JSON files to a dataframe. But first, you need to gain access to the API!

2.6.1 Getting your first Census API Key (unless you already have one)

In order to access the Census API, you will need your own Census API key. I could give you mine, but it's generally not good practice to share API keys unless you trust the person. Why? (1) If someone misuses your API key, you could get banned from accessing the API. (2) Especially for government or non-profit agencies, it can be a good idea to have your own so that they can keep track of how many users they have. Not all API access is free, but for government, non-profit, or institutionally held data, access is free although you may have different hoops to jump through for access depending on how secure they want to keep the data. (e.g., to protect survey respondents).

Question 9 [1 point]:

You will sign up for your API census key here (https://api.census.gov/data/key_signup.html). You should be able to obtain your key very quickly, but if you run into trouble then I will give you the data I have downloaded through **my** API key so that you can still keep going with the project.

Once your have your key, fill it into this code chunk here:

```
## Set your API key (put it between the quotation marks):
key <- "9738bbecc34a660d23499bf57d0c7d0e99b11d84"
```

2.6.2 Downloading data tables from the 2020 Census API

You are going to use the 2020 Census since that is the last year of data you have for the mortality dataset. The U.S. Census maintains numerous tables you can use for data. You will be using the **Decennial Census: Demographic Profile**. You can find the variable explanations for this table of the 2020 Census data here (%22<https://api.census.gov/data/2020/dec/dp/variables.html>%22). Below is a custom function I have written that will allow you to extract any of the variables you want from this table as long as you follow the usage.

HOW TO GET DATA FROM AN API: THE `url`

The important thing to understand about APIs is that they are based on the `url` to the API endpoint. For example,

`https://api.census.gov/data/2020/dec/dp?get=NAME,DP1\_0001C,DP1\_0025C&for=county:\*&in=state:06&key=9738bbecc34a660d23499bf57d0c7d0e99b11d84`

is a `url` that you can break down into its constituent parts like this:

Component	Description
<code>https://api.census.gov/data</code>	Base API endpoint
<code>/2020/dec/dp</code>	Specific dataset path (2020 Decennial Census, Data Profile)
<code>?get=NAME,DP1_0001C,DP1_0025C</code>	Variables to retrieve (NAME and 2 demographic variables, DP1_0001C and DP1_0025C)
<code>&for=county:*</code>	Requesting data for all counties
<code>&in=state:06</code>	Restricting to California (FIPS code 06)
<code>&key=YOUR_API_KEY</code>	Your personal API key (required for access)

You can practice modifying this `url` by adding your personal key to the end. If you do, what you'll see is something is this if you copy and paste the **completed url** into your browser:

```
[{"NAME", "DP1_0001C", "DP1_0025C", "state", "county"},
{"Alameda County, California", "1682353", "824426", "06", "001"},
{"Alpine County, California", "1204", "633", "06", "003"},
{"Amador County, California", "40474", "22067", "06", "005"},
{"Butte County, California", "211632", "104641", "06", "007"},
{"Calaveras County, California", "45292", "22754", "06", "009"},
{"Colusa County, California", "21839", "10996", "06", "011"},
{"Contra Costa County, California", "1165927", "568066", "06", "013"},
{"Del Norte County, California", "27743", "15039", "06", "015"},
{"El Dorado County, California", "191185", "94829", "06", "017"},
{"Fresno County, California", "1008654", "501441", "06", "019"},
{"Glenn County, California", "28917", "14488", "06", "021"},
{"Humboldt County, California", "136463", "67754", "06", "023"},
{"Imperial County, California", "179702", "90121", "06", "025"},
{"Inyo County, California", "19016", "9616", "06", "027"},
{"Kern County, California", "909235", "460227", "06", "029"},
{"Kings County, California", "152486", "82573", "06", "031"},
{"Lake County, California", "68163", "34509", "06", "033"},
{"Lassen County, California", "32730", "20720", "06", "035"},
{"Los Angeles County, California", "10014009", "4906513", "06", "037"},
{"Madera County, California", "156255", "77881", "06", "039"},
{"Marin County, California", "262321", "128593", "06", "041"},
{"Mariposa County, California", "17131", "8609", "06", "043"},
{"Mendocino County, California", "91601", "45837", "06", "045"},
{"Merced County, California", "281202", "140337", "06", "047"},
{"Modoc County, California", "8700", "4347", "06", "049"},
{"Mono County, California", "13195", "7048", "06", "051"},
{"Monterey County, California", "439035", "221123", "06", "053"},
{"Napa County, California", "138019", "67777", "06", "055"},
{"Nevada County, California", "102241", "50424", "06", "057"},
{"Orange County, California", "3186989", "1563452", "06", "059"},
{"Placer County, California", "404739", "196942", "06", "061"},
{"Plumas County, California", "19790", "9948", "06", "063"},
{"Riverside County, California", "2418185", "1199118", "06", "065"},
{"Sacramento County, California", "1585055", "774787", "06", "067"},
{"San Benito County, California", "64209", "31941", "06", "069"},
{"San Bernardino County, California", "2181654", "1078849", "06", "071"},
{"San Diego County, California", "3298634", "1642796", "06", "073"},
{"San Francisco County, California", "873965", "446144", "06", "075"},
{"San Joaquin County, California", "779233", "388163", "06", "077"},
{"San Luis Obispo County, California", "282424", "140271", "06", "079"},
{"San Mateo County, California", "764442", "377546", "06", "081"},
{"Santa Barbara County, California", "448229", "222051", "06", "083"},
{"Santa Clara County, California", "1936259", "975313", "06", "085"},
{"Santa Cruz County, California", "270861", "134471", "06", "087"},
{"Shasta County, California", "182155", "89654", "06", "089"},
{"Sierra County, California", "3236", "1635", "06", "091"},
{"Siskiyou County, California", "44076", "21872", "06", "093"},
{"Solano County, California", "453491", "224784", "06", "095"},
{"Sonoma County, California", "488863", "238535", "06", "097"},
{"Stanislaus County, California", "552878", "272558", "06", "099"},
{"Sutter County, California", "99633", "49585", "06", "101"},
{"Tehama County, California", "65829", "32827", "06", "103"},
{"Trinity County, California", "16112", "9068", "06", "105"},
{"Tulare County, California", "473117", "234710", "06", "107"},
{"Tuolumne County, California", "55620", "28790", "06", "109"},
{"Ventura County, California", "845843", "414483", "06", "111"},
{"Yolo County, California", "216403", "104292", "06", "113"},
{"Yuba County, California", "81575", "41063", "06", "115"}]
```

This is the Census data for the two variables chosen in JSON format! You can then convert these into a dataframe using the `fromJSON()` function from the `jsonlite` library, although this is not the only JSON handling package that exists in R. Similar packages exist in Python.

HOW THE FUNCTION WORKS

The custom function, `getPopSizes()`, pulls population data from the U.S. Census API for all counties in California, specifically demographic details like total population, gender counts, and age 65+ breakdowns from the 2020 Decennial Census.

Before you run the function, you must set the stage:

- `state = "06"`
 - Sets the state FIPS code for California (FIPS 06 = CA)
 - If you wanted all states, you'd use `"**"`
- `vars`
 - A vector of Census variable codes for population totals, age, and sex breakdowns
 - For example, `DP1_0001C` is total population
- `header`
 - A human-readable list of column names that will replace the technical variable codes in `vars`

Question 10A [0.5 points]:

Using a search engine or the Census table explorer (<https://data.census.gov>), whichever you find easier to use, look for the differences between table variables `DP1_0048C` and `DP1_0048P`. What different information do these two variables hold?

The variables `DP1_0048C` and `DP1_0048P` describe the same demographic characteristic, but they report it in different ways. `DP1_0048C` represents the count (the actual number of people in that category), while `DP1_0048P` represents the percentage of the population that falls into that category. The count variable provides the raw number of individuals, whereas the percent variable standardizes the measure relative to the total population. Percentages are often more useful when comparing counties of different population sizes because they are not influenced by the overall population count.

Digging into the Function

The function `getPopSizes()` takes in the following information as **arguments** to be able to run correctly:

- `key` : your Census API key
- `vars` : variable codes
- `header` : column names you want

Then, inside the function, it is doing several key tasks:

1. Builds the `url` for the API request
 - Adds variables to `get=...`
 - Limits to counties in California via `for=county:*&in=state:06`
 - Appends the API key at the end of the `url`
2. Requests the data from the Census API using `fromJSON(url)` and converts it to a data frame
3. Cleans the data for you
 - Removes the first row (which contains column names from the API)
 - Renames columns using `header`
 - Converts character columns back into numbers (`mutate_at`)
4. Returns a cleaned data frame of all the census variables requested, complete with human-readable headers!
 - This dataframe is now ready to be joined with your mortality shapefile!

Function call This line runs the function using your API key and saves the result to a variable called `census`: `census <- getPopSizes(key, vars, header)`

NOTE Make sure to change `include=FALSE` to `TRUE` in the first line (the `{r}` setup line) the chunk or you will get an error upon knitting.

```
## Set the state want you in double-digit FIPS; if you want all states, use *
state = "06" ## California

## Set a vector of variables you want; no need to include NAME,
## it's already written into the function
vars <- c("DP1_0001C", "DP1_0025C", "DP1_0049C", "DP1_0024C",
          "DP1_0024P", "DP1_0048C", "DP1_0048P", "DP1_0072C",
          "DP1_0072P")

## Set my header
header <- c("totalN", "malesN", "femalesN", "over65", "over65perc",
            "males65plus", "males65plusPerc", "females65plus",
            "females65plusPerc")

getPopSizes <- function(key, vars, header) {
  ## Building your url

  ## initialize "get" variable
  get <- "NAME"
  ## For each of the variable, add in the codes to the "get" variable so you can
  ## add that into the API address
  for(i in 1:length(vars)){
    get <- paste(get, vars[i], sep="&")
  }
  ## Add in more of the necessary pieces into the API address
  for.statemt <- paste0("&for=county:*&in=state:", state)
  base.url <- "https://api.census.gov/data/2020/dec/dp"
  ## The finished URL that you will use to get the JSON file
  url <- paste0(base.url, "?get=", get, for.statemt, "&key=", key)

  ## Use fromJSON() to get the data table and turn into a data frame
  census <- data.frame(fromJSON(url))
  census <- census[-1, ] ## first row needs to be dropped

  ## Rename the columns with what you want
  names(census) <- c("NAME", header, "STATE", "COUNTY")

  ## Fix the fact that everything numeric was turned into character
  census <- census %>%
    mutate_at(header, as.numeric)

  ## Done!
  return(census)
}

## Call the function
census <- getPopSizes(key, vars, header)
## Remove the extra characters from the name of the county
census <- census %>%
  mutate(NAME = gsub(" County, California", "", NAME)) %>%
  rename(County = NAME)
```

Did you notice how quickly it accessed the information? That is one of the strengths of APIs. You also get to customize the tables to only grab what you want and you can imagine how you might iterate over multiple years or other structures using a function similar to what I coded for you above. APIs are wonderful to become familiar with as a data scientist; I highly recommend practicing more with APIs as you move toward the job market as it is a highly sought after skill! I also strongly recommend the R package `tidycensus` (<https://walker-data.com/tidycensus/>); I did not choose to use that here so you would have **generic code** you could use for many API endpoints going forward in your career! Additionally, `tidycensus` is restrictive since it only access some tables of the Census (notably the American Community Survey or ACS), but you were using a different table here.

Question 10B [0.5 points]:

Use code or some other method to explore the resulting dataframe, `census`. Explain what this dataframe contains.

```
glimpse(census)
```

```
## Rows: 58
## Columns: 12
## $ County      <chr> "Alameda", "Alpine", "Amador", "Butte", "Calaveras",...
## $ totalN      <dbl> 1682353, 1204, 40474, 211632, 45292, 21839, 1165927,...
## $ malesN      <dbl> 824426, 633, 22067, 104641, 22754, 10996, 568066, 15...
## $ femalesN     <dbl> 857927, 571, 18407, 106991, 22538, 10843, 597861, 12...
## $ over65      <dbl> 240483, 301, 11480, 40155, 12762, 3251, 192799, 5241...
## $ over65perc   <dbl> 14.3, 25.0, 28.4, 19.0, 28.2, 14.9, 16.5, 18.9, 22.3...
## $ males65plus  <dbl> 106747, 160, 5712, 18494, 6220, 1536, 85071, 2507, 2...
## $ males65plusPerc <dbl> 12.9, 25.3, 25.9, 17.7, 27.3, 14.0, 15.0, 16.7, 21.6...
## $ females65plus <dbl> 133736, 141, 5768, 21661, 6542, 1715, 107728, 2734, ...
## $ females65plusPerc <dbl> 15.6, 24.7, 31.3, 20.2, 29.0, 15.8, 18.0, 21.5, 23.0...
## $ STATE       <chr> "06", "06", "06", "06", "06", "06", "06", "06"...
## $ COUNTY      <chr> "001", "003", "005", "007", "009", "011", "013", "01..."
```

The census dataframe contains demographic information for all 58 California counties obtained from the 2020 Decennial Census API. Each row represents a county, and the 12 columns contain population characteristics and county identifiers. The dataframe includes the total population (`totalN`), male population (`malesN`), female population (`femalesN`), the number of residents age 65 and older (`over65`), and the percentage of residents age 65 and older (`over65perc`). It also includes counts and percentages of males and females age 65 and older, along with state and county FIPS codes. The population sizes vary substantially across counties. For example, Alameda County has a population of over 1.6 million people, while Alpine County has a population of just over 1,200 people. This population information is important because it can be joined with the mortality data to calculate population-adjusted mortality measures rather than relying solely on total death counts.

Question 11 [1 point]:

Now that you know more generally how this function works and have seen it in action, in your own words, explain how you could adapt this approach (or even this function!) to work with a different API (not the Census). I am not asking you to code, but I am asking you to think through:

1. What key pieces of the function would you need to change?
2. What general steps stay the same regardless of the API?

- **Hint 1:** Think about how the `url` is built and what makes it specific to the Census API.
- **Hint 2:** Think about how the data is retrieved (e.g., using `fromJSON()`) and cleaned afterward.
- **Hint 3:** For simplicity, assume that the data retrieved are in `JSON` format, since that is the most commonly used format.

To adapt this function for a different API, we would first need to change the parts of the function that are specific to the Census API. This would include changing the base URL, the endpoint path, the query parameters, and the variable names being requested. We would also need to check whether the new API requires an API key and how that key should be included in the request. For example, some APIs include the key at the end of the URL, while others require it in a header or token. The general process would stay mostly the same. We would still build a URL for the API request, specify the variables or fields we want to retrieve, request the data using a function such as `fromJSON()`, convert the JSON response into a dataframe, and then clean the result. The cleaning step would depend on the structure of the API response, but it would likely include renaming columns, removing unnecessary rows or nested elements, and converting variables to the correct data types. Overall, the Census-specific details would change, but the general workflow would remain the same: build the API request, retrieve the JSON data, convert it into a usable dataframe, clean the dataframe, and return the final dataset for analysis.

2.6.3 Joining the census with the spatial data and updating the maps

Question 12 [3 points]:

Now, it's your turn to do some work! Your task is to join the census data to the shapefile of overall mortality we've already made. You want a tibble that contains all of the data from the census dataframe and all of the data from `mortality_sf`. I called mine `mortality_sf2`, so you will want to call yours the same unless you plan to change the variable name everywhere else downstream.

- **Hint 1:** The easiest way is to use the FIPS data to join.
- **Hint 2:** Don't worry about calculating % mortality yet. I will help you with that below.
- **Hint 3:** If you're having problems, check your data types!

```
head(mortality_sf$COUNTY_NUM)
```

```
## [1] 1 1 1 1 1 1
```

```
head(census$COUNTY)
```

```
## [1] "001" "003" "005" "007" "009" "011"
```

```
mortality_sf2 <- mortality_sf %>%
  #Join by county name because this produced a complete match between
  ## the mortality shapefile and Census dataframe
  left_join(census, by =c("COUNTY_NAME" = "County")) %>%
  st_as_sf()
```

```
sum(is.na(mortality_sf2$totalN))
```

```
## [1] 0
```

```
glimpse(mortality_sf2)
```

```
## Rows: 2,596
## Columns: 19
## Groups: COUNTY_NAME, Year [232]
## $ COUNTY_NAME      <chr> "Alameda", "Alameda", "Alameda", "Alameda", "Alameda..."
## $ Year              <int> 2017, 2017, 2017, 2017, 2017, 2017, 2017, 2017, 2017...
## $ Month            <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 1, 2, 3, 4, 5...
## $ totalDeaths      <int> 1048, 831, 880, 859, 852, 772, 807, 750, 769, 811, 7...
## $ COUNTY_NUM       <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1...
## $ GlobalID         <chr> "6341ecf5-b0ec-4a0c-94dc-05b8b0505dae", "6341ecf5-b0...
## $ CountyStri       <chr> "1", "1", "1", "1", "1", "1", "1", "1", "1", "1", "1...
## $ geometry         <POLYGON [m]> POLYGON ((-13611182 4566017..., POLYGON ((-1...
## $ totalN           <dbl> 1682353, 1682353, 1682353, 1682353, 1682353, 1682353...
## $ malesN           <dbl> 824426, 824426, 824426, 824426, 824426, 824426, 8244...
## $ femalesN         <dbl> 857927, 857927, 857927, 857927, 857927, 857927, 8579...
## $ over65           <dbl> 240483, 240483, 240483, 240483, 240483, 240483, 2404...
## $ over65perc       <dbl> 14.3, 14.3, 14.3, 14.3, 14.3, 14.3, 14.3, 14.3, 14.3...
## $ males65plus      <dbl> 106747, 106747, 106747, 106747, 106747, 106747, 1067...
## $ males65plusPerc  <dbl> 12.9, 12.9, 12.9, 12.9, 12.9, 12.9, 12.9, 12.9, 12.9...
## $ females65plus    <dbl> 133736, 133736, 133736, 133736, 133736, 133736, 1337...
## $ females65plusPerc <dbl> 15.6, 15.6, 15.6, 15.6, 15.6, 15.6, 15.6, 15.6, 15.6...
## $ STATE            <chr> "06", "06", "06", "06", "06", "06", "06", "06", "06"...
## $ COUNTY           <chr> "001", "001", "001", "001", "001", "001", "001", "001", "00..."
```

Note: If you're unable to do this for any reason, you can pick back up here. You will need to download the data from Google Drive (<https://drive.google.com/file/d/15MT127zoySDS7kOf9uhLwPSIWBCok1W/view?usp=sharing>), as the shapefile is too large for GitHub yet again. I have placed mine on my Desktop, but you can put it anywhere you like and update the directory path. Just make sure not to put it in your GitHub repo!

```
## Comment me out or change EVAL=FALSE to run your own versions!
#load("~/Desktop/mortality_sf2.Rdata")
#load("census.Rdata")
```

2.7 Mapping population-adjusted mortality

Now, at this point, you were so excited to have these data that you decide to next calculate the annual population-adjusted % mortality per county per year so that you could recreate the map you tried to make above. **Notice** that, this time, I set the color scale for the chloropeth to equal the **five number summary** from `fivenum()` so that you could more easily see the median versus the quartiles.

NOTE: If you get errors here, it is because your `mortality_sf2` object is not quite right!

```
##### Calculate the annual percent mortality
mortalitySummary <- mortality_sf2 %>%
  ## Group by County as well as Year
  group_by(COUNTY_NAM, Year) %>%
  ## Summarize by calculating an annual percent mortality
  ## Note that I have to divide by the mean of the N from the Census
  ## or I'd get an error
  summarize(annualPercMortality = 100*(sum(totalDeaths,
                                           na.rm = TRUE)/mean(totalN))) %>%

  ## Ungroup - necessary to remove the grouping for future use
  ungroup()

##### Find the five number summary
summ <- fivenum(mortalitySummary$annualPercMortality)
paste0("The five-number summary of annual % total mortality is: ")
```

```
## [1] "The five-number summary of annual % total mortality is: "
```

```
paste(c("Min", "Lower Quartile", "Median", "Upper Quartile", "Max"),
      round(summ, 3))
```

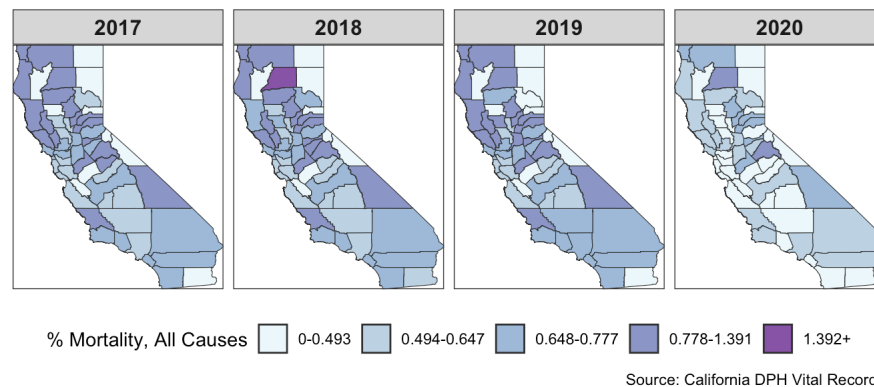
```
## [1] "Min 0" "Lower Quartile 0.503" "Median 0.652"
## [4] "Upper Quartile 0.781" "Max 1.392"
```

```
##### Now add on the cuts and labels for mapping:
mortalitySummary <- mortalitySummary %>%
  mutate(deathLabels = cut(annualPercMortality,
                          breaks = c(summ, Inf),
                          labels = c("0-0.493", "0.494-0.647", "0.648-0.777",
                                     "0.778-1.391", "1.392+"),
                          ordered_result = TRUE, right = F))
```

Your population-adjusted map now shows the **percent mortality** in the population rather than total numbers of deaths. In other words, **per capita**, what percentage of total deaths were there each year?

California Annual Percent Mortality by County (All Causes)

2017 - 2020



For example, for 3 of the 4 years, **Shasta County** (2017-2019) shows the **highest percent mortality** at over 1.39%.

Question 13 [1 points]:

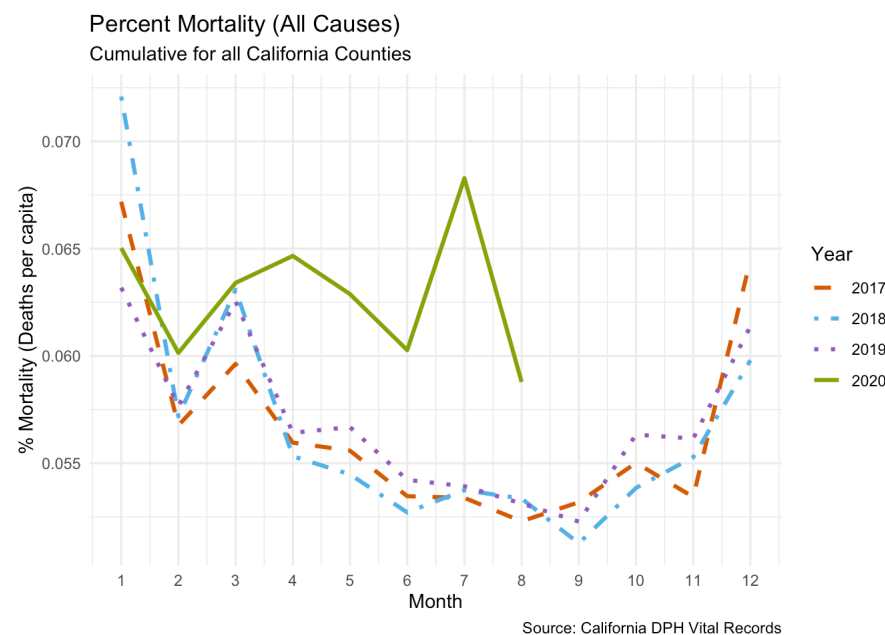
What patterns do you observe now, if any? Do you think this is a true pattern? Aren't you being told that deaths are going **up** due to COVID-19??

After adjusting for population size, we notice a very different pattern than what we saw with total deaths. Los Angeles County is no longer the clear standout, which suggests that much of the pattern in the earlier maps was being driven by population size rather than mortality risk itself. Instead, counties such as Shasta consistently appear in the highest mortality category from 2017 through 2019. One thing that stands out is that we do not see a dramatic statewide increase in percent mortality in 2020, even though this was the first year of the COVID-19 pandemic. At first, this seems surprising because we know deaths increased during COVID-19. However, this map is showing annual percent mortality grouped into broad categories rather than raw death counts. Small increases in mortality may therefore not be large enough to move counties into a different category. In addition, COVID only affected part of 2020, and we are looking at all-cause mortality rather than COVID-specific mortality. Based on these maps, we would be cautious about concluding that COVID had no impact on mortality. Instead, the results suggest that county-level mortality patterns remained relatively stable over time and that additional analyses would be needed to better understand the effects of the pandemic.

2.8 The data for 2020 isn't complete...

Ohhhh... that's right. The pattern observed is simply a by-product of an incomplete record from 2020, isn't it?! You only have 8 months' of observations. You realize at this point you were so excited to jump into map-making that you missed something in how you were doing your calculations and/or planning your maps.

You decide that you had better take a look at both annual and monthly trends in the total numbers of deaths to see if that sheds some light on things:



Question 14 [1 point]:

Why did this happen and how could you fix it? (**There is more than one correct answer possible.**) Could you have made your maps the way you did in Section 2.7 and had them be accurate if we'd had a full 12 months' of data for each year?

This happened because the 2020 data are incomplete. Looking at the line graph, we can see that 2020 only contains data through August, while the other years contain all 12 months. As a result, when we calculated annual percent mortality in Section 2.7, we were comparing a partial year of data against full years. This would make the 2020 mortality percentages appear artificially lower than they actually were. One way to fix this would be to only compare the first eight months of every year so that all years are based on the same time period. Another option would be to exclude 2020 from the annual comparison until a complete year of data is available. We could also recalculate the annual summaries once the missing months are added. Yes, we could have made the maps in Section 2.7 and had them be accurate if we had all 12 months of data for each year. The issue was not with the mapping approach itself, but with the fact that the 2020 mortality data were incomplete, which led to an unfair comparison across years.

3.1 Fixing the map's accuracy for 2020.

Although there are several ways you could tackle this, what I show you below is how to take an average of monthly % mortality so that it no longer matters whether you have 8 months' or 12 months' of data available for any given year. **NOTE** that how you accomplish this is by grouping by county, year, AND month; this way, by looking at a monthly average of percent mortality, it no longer matters whether you have an entire year's data for 2020 or not!

```
##### Calculate the monthly percent mortality
mortalitySummary <- mortality_sf2 %>%
  ## Group by County as well as Year
  group_by(COUNTY_NAM, Year, Month) %>%  ## <- WHERE A KEY CHANGE HAPPENS!
  ## Summarize by calculating a monthly percent mortality
  ## Note that I have to divide by the mean of the N from the Census
  ## or I'd get an error
  summarize(monthlyPercMortality = sum(totalDeaths,
                                       na.rm = TRUE)/mean(totalN)*100) %>%
  ## Ungroup - necessary to remove the grouping for future use
  ungroup() %>%
  ## NOW, you can group again by county and year and calculate the true annual
  ## average that has been adjusted for differing number of months!
  group_by(COUNTY_NAM, Year) %>%  ## <- no Month now!
  summarize(annualPercMortality = mean(monthlyPercMortality, na.rm = TRUE)*12)

##### Find the five number summary
summ <- fivenum(mortalitySummary$annualPercMortality)
paste0("The five-number summary of annual % total mortality is: ")
```

```
## [1] "The five-number summary of annual % total mortality is: "
```

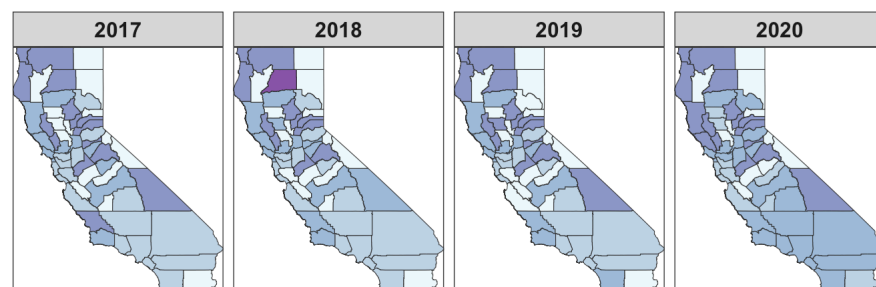
```
paste(c("Min", "Lower Quartile", "Median", "Upper Quartile", "Max"),
      round(summ, 3))
```

```
## [1] "Min 0"          "Lower Quartile 0.578" "Median 0.694"
## [4] "Upper Quartile 0.868" "Max 1.392"
```

```
##### Now add on the cuts and labels for mapping:
mortalitySummary <- mortalitySummary %>%
  mutate(deathLabels = cut(annualPercMortality,
                          breaks = c(summ, Inf),
                          labels = c("0-0.571", "0.572-0.689", "0.690-0.861",
                                     "0.862-1.391", "1.392+"),
                          ordered_result = TRUE, right = F))
```

California Monthly Percent Mortality by County (All Causes)

2017 - 2020



% Mortality, All Causes 0-0.571 0.572-0.689 0.690-0.861 0.862-1.391 1.392+

Source: California DPH Vital Records

Question 15 [1 points]:

What patterns do you observe now in 2020? (**NOTE:** The maps for 2017-2019 are unchanged.) Does it now feel consistent with the idea that deaths could be increasing in 2020 compared to previous years? For which counties yes but others no?

Hint: Take a look at counties in Southern CA, like Los Angeles, Orange, San Diego, Riverside, Imperial...

After correcting for the fact that 2020 only contained eight months of data, the 2020 map appears much more consistent with what we would expect during the first year of the COVID-19 pandemic. Unlike the previous map, 2020 no longer appears artificially lighter than the earlier years. We notice that several counties in Southern California, including Los Angeles, Riverside, Imperial, and parts of San Diego and Orange County, appear to move into higher mortality categories in 2020. This suggests that mortality may have increased in these areas compared to previous years. At the same time, the pattern is not uniform across the entire state, as some counties show little change from 2017–2019. Overall, these results seem more realistic because COVID-19 did not affect every county equally. The increases observed in several Southern California counties are consistent with the idea that deaths were rising during 2020, while other counties may have experienced smaller changes or remained relatively stable.

3 Project future mortality

Your stakeholders have asked if you can quickly give them *any* idea of what to expect mortality to be across the counties for the remaining months of 2020. Now, even though you knew it would not be the most robust method for time series projection available (a **caveat!**), you immediately think to yourself, “Oh, I could do a regression.” So, you do.

3.1 Regression predictions of mortality in 2020

First, you need to do a little prep to the data to get you started. You need to calculate the monthly percent mortality and attach it to your mortality shapefile, `mortality_sf2`.

```
## First, calculate monthly percent mortality in your dataframe:
mortality_sf2 <- mortality_sf2 %>%
  ## Group by County as well as Year
  group_by(COUNTY_NAM, Year, Month) %>%
  ## Summarize by calculating a monthly percent mortality
  ## Note that I have to divide by the mean of the N from the Census
  ## or I'd get an error
  summarize(monthlyPercMortality = sum(totalDeaths,
                                       na.rm = TRUE)/mean(totalN)*100) %>%
  ## Ungroup - necessary to remove the grouping for future use
  ungroup()
```

Now, fit the OLS linear regression.

```
olsModel <- lm(monthlyPercMortality ~ COUNTY_NAM + Year + Month, data = mortality_sf2)
```

Next, make some new data (stored into `newdata`) so you can generate predictions for those new data.

```
## Make the new data:
## Get the names of the counties; repeat four times for the 4 months
COUNTY_NAM <- rep(unique(mortality_sf2$COUNTY_NAM), 4)
## Make a set of months for each of the 58 counties for Sept, Oct, Nov, Dec
Month <- c(rep(9, 58),
           rep(10, 58),
           rep(11, 58),
           rep(12, 58))
Year <- rep(2020, length(Month))
## Put this into a new dataframe called "newdata"
newdata <- cbind.data.frame(COUNTY_NAM, Year, Month)
```

Now, generate the predictions using `newdata` and attach the predicted monthly percent mortality to `newdata`. This is necessary for plotting to occur! Make sure you **check out the dimensions of the `newdata`**. Since you are missing four months' of data (September - December) in 2020, there should be a row for every county for every month missing.

```
dim(newdata)
```

```
## [1] 232 3
```

```
## Attach it to the dataframe and then call it the same name as the original Y
newdata$percMortality <- predict(olsModel, newdata = newdata)
## Also, add a flag so that you know it is predicted
newdata$isPredicted <- "1"
```

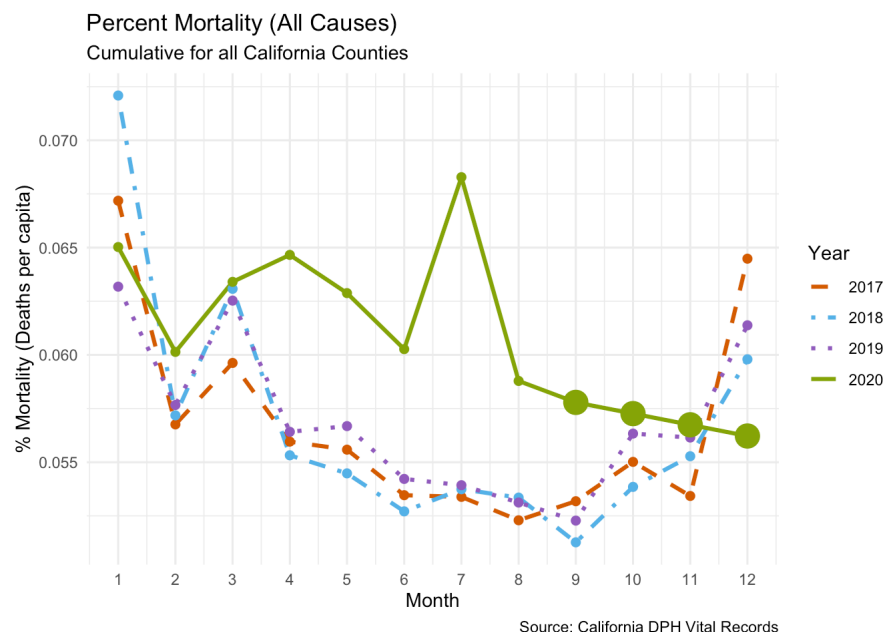
Finally, you are ready to make a temporary dataframe, which you will call `temp`, which will pull everything together - the original data and the predictions - so that you can add them to your line graphs from above. Although I could choose to add the predictions to your mortality shapefile instead, I am choosing not to in order to keep that more pristine. I don't want to add predictions that I may not use in the future to my good dataframe!

```
temp <- totalDeaths %>% ## Start from totalDeaths for simplicity
  ## Join with census by County
  full_join(census, by = "County") %>%
  ## Group by Year and then Month to aggregate the counties
  group_by(Year, Month) %>%
  ## Calculate the percent mortality - it is a percent because multiply by 100
  summarise(percMortality = sum(totalDeaths, na.rm = TRUE) /
    sum(totalN, na.rm = TRUE)*100) %>%
  ## Keep only the columns you need
  select(Year, Month, percMortality) %>%
  ## Make an empty flag column filled with zeros to flag if the data are
  ## predicted
  mutate(isPredicted = "0")

## Clean up newdata a little: you need to aggregate across the counties by year
newdata <- newdata %>%
  ## Group by Year and then Month to aggregate the counties; add isPredicted
  ## in there too so that it makes it into the final dataset!
  group_by(Year, Month, isPredicted) %>%
  ## Remove what you don't need
  select(-COUNTY_NAM) %>%
  ## Calculate a mean percent mortality
  summarize(percMortality = mean(percMortality, na.rm = TRUE))

## Row bind newdata onto temp to give you a complete, ordered, dataset for plots!
temp <- rbind(temp, newdata)
```

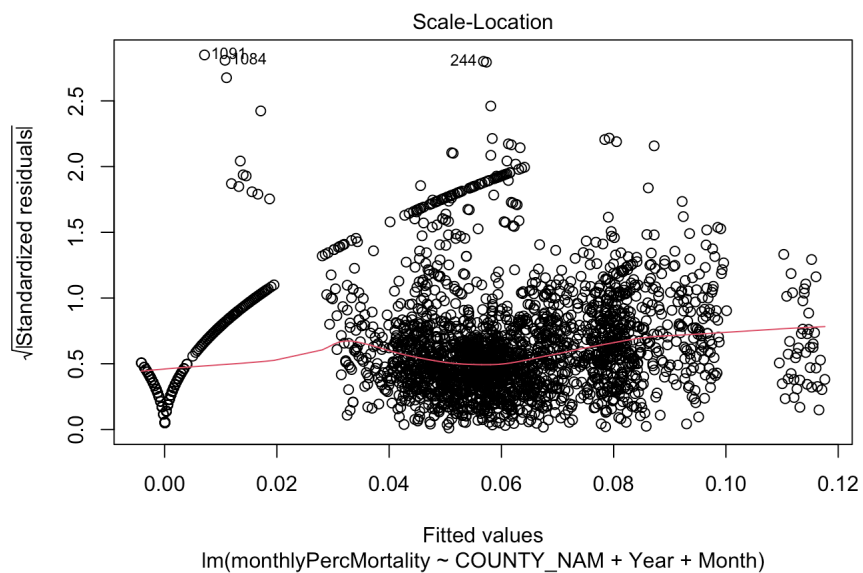
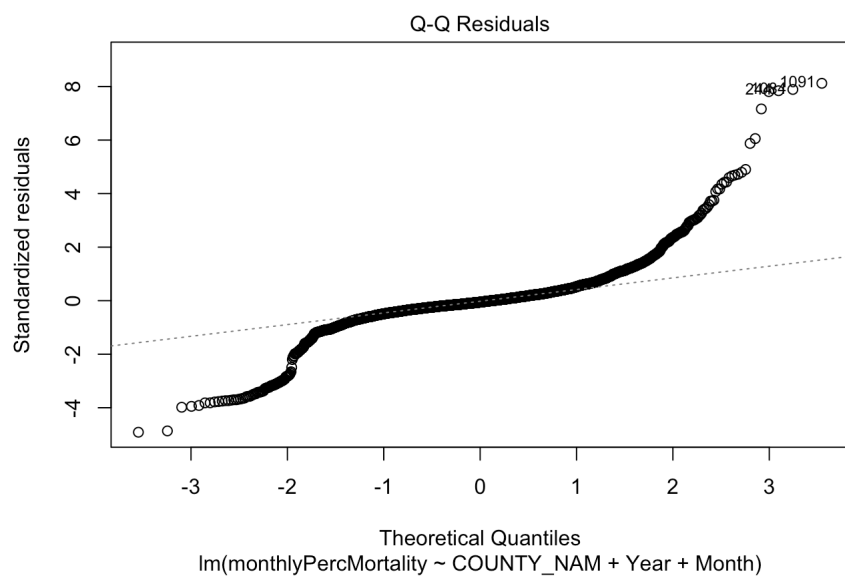
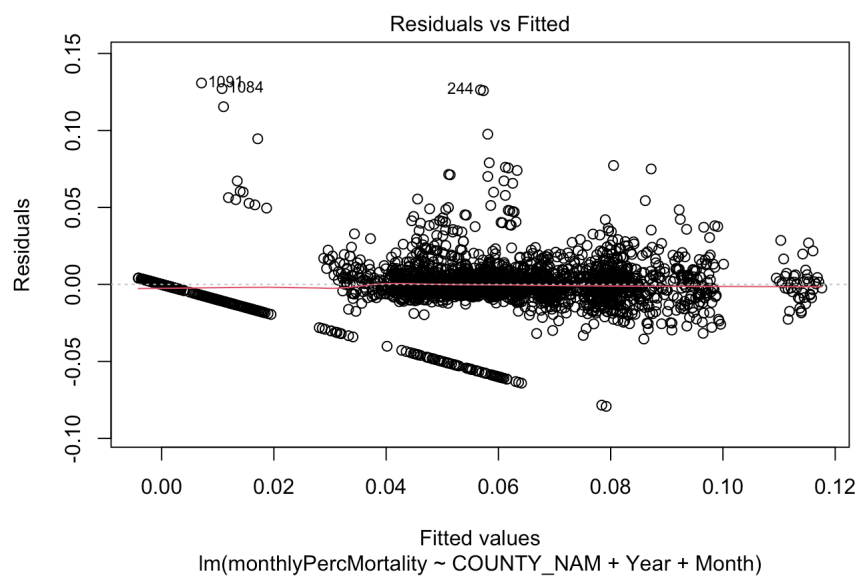
Lastly, finish out the year of 2020 with predictions (shown in bubble points):

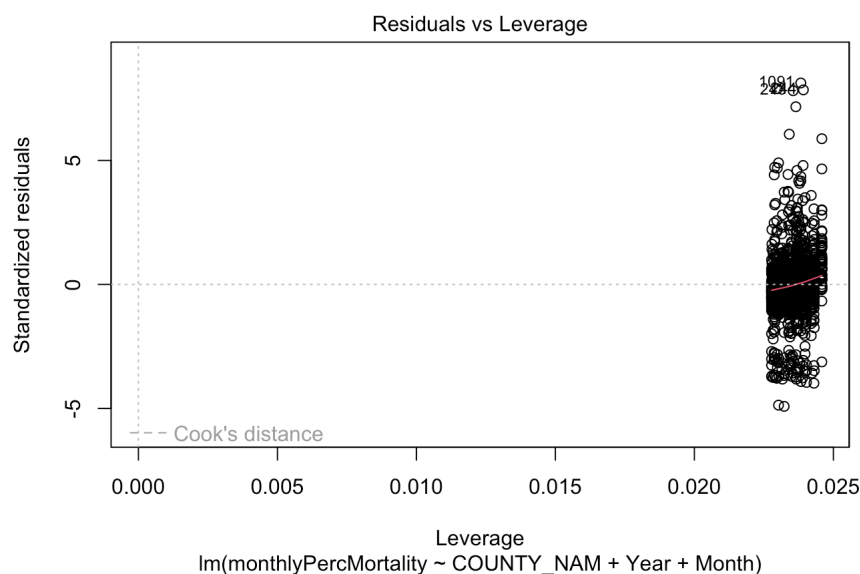


Question 16 [1 point]:

Do you think this regression prediction is robust? Why or why not? (Hint: Check your assumptions with `plot(olsModel)` !!)

```
plot(olsModel)
```





We do not think this regression prediction is particularly robust. Looking at the diagnostic plots, several of the assumptions of OLS appear to be violated. The Q-Q plot shows that the residuals are not normally distributed, as there is substantial deviation from the reference line and several extreme observations in the tails. The Scale-Location plot also suggests that the variance of the residuals may not be constant across fitted values, indicating potential heteroscedasticity. In addition, the Residuals vs Leverage plot shows a few observations with relatively large residuals that may be influencing the model. Beyond the diagnostic plots, the model itself is fairly simple because it only uses county, year, and month to predict mortality. It does not account for seasonality, spatial effects, COVID-19 outbreaks, demographic differences, or other factors that could influence mortality. As a result, the predictions may provide a rough estimate of future mortality, but we would be cautious about treating them as highly accurate forecasts.

3.2 Spatial lag regression

Then you recall that class you took during your Master's degree - what was it called again? Doesn't matter! You remember that calculating **spatial weights** would allow you to do a type of spatial regression, such as spatial lag regression, because it deals with **spatial autocorrelation**. Right, that was the term!

3.2.1 Calculate spatial weights matrix

The first step in calculating a spatial weights matrix is to ensure geometry is retained in the shapefile, even if you plan to aggregate by month or by year for mapping. I actually had you covered earlier when you first calculated the monthly percent mortality, `monthlyPercMortality`, in the **mortality shapefile**, `mortality_sf2`. By ungrouping at the end of the calculation, I ensured that you lost no data or geometry! This is also exactly WHY you did not do any aggregation directly on these data or attach the subpar predictions to `mortality_sf2`.

Warning: Careless aggregation WILL rob you of spatial or other information, which will make spatial modeling impossible. This is because, in order to run spatial models, the spatial object *must* retain geometry.

Now that you know you have retained all of your spatial information in `mortality_sf2`, you can create the spatial weights matrix similarly to how you did it in Demo 2 using `spdep`.

1. Create neighbor list using Queen contiguity. *This could take a minute to run, be patient.*

*This identifies which counties touch each other (using `poly2nb()`). Note, however, that you are only looking at the **unique pairs** of county names and geometries, however, for this step.*

```
mortality_nb <- poly2nb(mortality_sf2 %>%
  ## Isolate the unique county name-geometry pairs
  select(COUNTY_NAM, geometry) %>% unique() %>%
  ## Arrange the county names alphabetically (helps
  ## ensure no downstream issues)
  arrange(COUNTY_NAM) %>%
  ## Turns the county name into a factor (helps ensure
  ## no downstream issues)
  mutate(COUNTY_NAM = as_factor(COUNTY_NAM)))

## Make sure the size matches the number of counties:
length(mortality_nb)
```

```
## [1] 58
```

Question 17A [0.5 points]

Why am I calculating the neighbors on just the **distinct counties** in the mortality shapefile when I just finished saying you don't want to lose any of your spatial data for the model?

Hint 1: What are the dimensions of `mortality_sf2` and is it a multiple of 58?

Hint 2: What happens if I calculate spatial weights on redundant data? Does it cost me computationally?

We calculated the neighbor list using only the distinct counties because the spatial relationships between counties do not change across months or years. Although `mortality_sf2` contains 2,596 observations, there are only 58 unique counties, and each county appears repeatedly for different months and years. Calculating neighbors on all 2,596 rows would repeatedly calculate the same county boundaries and neighbor relationships many times. By using only the unique county geometries, we retain all of the spatial information needed for the model while avoiding redundant calculations. This makes the process much more computationally efficient without losing any meaningful geographic information. The spatial weights matrix only needs to know which counties border each other once, not for every month and year in the dataset.

2. Calculate row-standardized spatial weights matrix based on the neighbors-list. This converts the neighbor information stored in `mortality_nb` into a usable weight list using `nb2listw()`. Neighbors have equal influence (row-standardized: `style = "W"`).

```
mortality_sw <- nb2listw(mortality_nb, style = "W", zero.policy = TRUE)
```

3.2.2 Join spatial weights to the mortality shapefile so that every datapoint has its spatial weight.

You want to calculate, for each county in each year and month, the average percent mortality of its neighboring counties (i.e., the spatial lag). This helps you model whether mortality in one county is influenced by nearby counties. Recall from lecture and Demo 2 that a **spatial lag** is the weighted average of a variable (here, mortality) in neighboring counties. For example, if Los Angeles county is surrounded by five counties, the spatial lag for L.A. county is the average mortality rate across those five neighbors, adjusted by the spatial weights.

To accomplish this, what you need to do is loop over the time components (year and then month), and compute spatial lag for within each of the four years. you will use `purrr` to do this; it's more computationally efficient than writing a more simple `for()` loop. As it iterates over the dataset, by year, it will use `spdep::lag.listw()` to calculate the spatial lag for each time point. The result is a new column in `mortality_sf2` that contains the average monthly mortality of any given county's neighbors for every county-month-year combination. **Don't forget!** You must center (not scale) the monthly percent mortality before computing the spatial weights.

```
## Make a centered transformation of monthly percent mortality
mortality_sf2$monthlyPercMortalityCentered <- scale(mortality_sf2$monthlyPercMortality,
                                                    center = TRUE, scale = FALSE)

## Initialize an empty vector to store spatial lag values
mortality_sf2$lagged_monthlyPercMortality <- NA_real_

## Calculate spatial weights for every time point
mortality_sf2 <- mortality_sf2 %>%
  arrange(COUNTY_NAM, Year, Month) %>% ## Ensures it is in same order as weights
  ## Handles each time slice separately
  group_by(Year, Month) %>%
  mutate(
    lagged_monthlyPercMortality = {
      ## Slice to get the current group's data (i.e., one month-year combo)
      ## and then sort within group to match spatial weights
      monYr <- arrange(cur_data(), COUNTY_NAM)$monthlyPercMortalityCentered
      ## Compute the spatial lag using the spatial weights in mortality_sw
      lag.listw(mortality_sw, monYr)
    }
  ) %>%
  ungroup()
```

3.2.3 Run spatial lag regression using the lagged monthly mortality as a covariate

In Demo 2, you used the `spatialreg` package's `lagsarlm()` function to perform a spatial lag regression. However, because you needed to compute the spatial lag within each year-month combo for each county, you have to perform your spatial lag regression a little differently using just the `lm()` function and explicitly using the spatially lagged monthly percent mortality ourselves.

Question 17B [0.25 points]

Run a **spatial lag regression (SAR)** model using the spatial lag you just calculated. Save the model as an object called `sarModel`. I have gotten it started for you so you use the spatial lag correctly, but you need to fill in the blanks, run the analysis, and assess the results.

Hint 1: Your target (dependent variable) is unchanged from OLS regression.

Hint 2: Make sure to include both time elements as covariates!

```
sarModel <- lm(monthlyPercMortality ~ COUNTY_NAM + Year + Month + lagged_monthlyPercMortality,
               data = mortality_sf2)

summary(sarModel)
```

```
##
## Call:
## lm(formula = monthlyPercMortality ~ COUNTY_NAM + Year + Month +
##     lagged_monthlyPercMortality, data = mortality_sf2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.078164 -0.004791 -0.000635  0.004438  0.133671
##
## Coefficients:
##              Estimate Std. Error t value
## (Intercept)    -1.14889429   0.61485161  -1.869
## COUNTY_NAMAlpine    -0.05166274   0.00344226 -15.008
## COUNTY_NAMAmador     0.03005154   0.00345834   8.690
## COUNTY_NAMButte      0.04409364   0.00344145  12.813
## COUNTY_NAMCalaveras   0.01414368   0.00344562   4.105
## COUNTY_NAMColusa     0.00802487   0.00344426   2.330
## COUNTY_NAMContra Costa 0.00321785   0.00344900   0.933
## COUNTY_NAMDel Norte   0.02673975   0.00360620   7.415
## COUNTY_NAMEl Dorado    0.00709148   0.00344204   2.060
## COUNTY_NAMFresno      0.01596718   0.00348766   4.578
## COUNTY_NAMGlenn       -0.01211036   0.00350468  -3.455
## COUNTY_NAMHumboldt     0.02504293   0.00345492   7.248
## COUNTY_NAMImperial    -0.00716570   0.00344148  -2.082
## COUNTY_NAMInyo        0.03350960   0.00349654   9.584
## COUNTY_NAMKern         0.00366564   0.00344359   1.064
## COUNTY_NAMKings       -0.00540637   0.00344145  -1.571
## COUNTY_NAMLake        0.02638432   0.00344432   7.660
## COUNTY_NAMLassen      -0.01448909   0.00349598  -4.144
## COUNTY_NAMLos Angeles  0.00475859   0.00344145   1.383
## COUNTY_NAMMadera      -0.00072696   0.00345449  -0.210
## COUNTY_NAMMarin       0.01187516   0.00344609   3.446
## COUNTY_NAMMariposa    -0.00075477   0.00345627  -0.218
## COUNTY_NAMMendocino    0.01876585   0.00344174   5.452
## COUNTY_NAMMerced      -0.01053147   0.00344266  -3.059
## COUNTY_NAMModoc       -0.04807154   0.00354426 -13.563
## COUNTY_NAMMono        -0.05099032   0.00344150 -14.816
## COUNTY_NAMMonterey    -0.00060520   0.00344689  -0.176
## COUNTY_NAMNapa        0.02503297   0.00345135   7.253
## COUNTY_NAMNevada      0.03067110   0.00344333   8.907
## COUNTY_NAMOrange      0.00493904   0.00344145   1.435
## COUNTY_NAMPlacer      0.02776556   0.00345894   8.027
## COUNTY_NAMPLumas      -0.00460813   0.00345474  -1.334
## COUNTY_NAMRiverside    0.00750452   0.00344145   2.181
## COUNTY_NAMSacramento   0.01373489   0.00344298   3.989
## COUNTY_NAMSan Benito  -0.01338139   0.00345429  -3.874
## COUNTY_NAMSan Bernardino 0.00751493   0.00344145   2.184
## COUNTY_NAMSan Diego    0.01128227   0.00348779   3.235
## COUNTY_NAMSan Francisco 0.00889673   0.00344155   2.585
## COUNTY_NAMSan Joaquin  0.00483960   0.00345483   1.401
## COUNTY_NAMSan Luis Obispo 0.02034828   0.00344748   5.902
## COUNTY_NAMSan Mateo    0.00204035   0.00344681   0.592
## COUNTY_NAMSanta Barbara 0.00950440   0.00344304   2.760
## COUNTY_NAMSanta Clara  -0.00139327   0.00344812  -0.404
## COUNTY_NAMSanta Cruz   0.00410392   0.00347483   1.181
## COUNTY_NAMShasta       0.06664008   0.00348880  19.101
## COUNTY_NAMSierra       -0.05215876   0.00344445 -15.143
## COUNTY_NAMSiskiyou     0.03875051   0.00344291  11.255
## COUNTY_NAMSolano       0.00990178   0.00344849   2.871
## COUNTY_NAMSonoma       0.01697884   0.00346679   4.898
## COUNTY_NAMStanislaus   0.02590326   0.00344294   7.524
## COUNTY_NAMSutter       -0.01110281   0.00348828  -3.183
## COUNTY_NAMTehama       0.01425534   0.00345167   4.130
## COUNTY_NAMTrinity      -0.04321781   0.00360070 -12.003
## COUNTY_NAMTulare       0.00043328   0.00344386   0.126
## COUNTY_NAMTuolumne     0.05001054   0.00351086  14.245
## COUNTY_NAMVentura      0.00473293   0.00344152   1.375
## COUNTY_NAMYolo         -0.00772675   0.00345271  -2.238
## COUNTY_NAMYuba         0.03294769   0.00344149   9.574
## Year                  0.00059573   0.00030458   1.956
## Month                -0.00038603   0.00009823  -3.930
## lagged_monthlyPercMortality 0.29223290   0.04068108   7.184
##
## Pr(>|t|)
```

```
## (Intercept) 0.061800 .
## COUNTY_NAMAlpine < 0.0000000000000002 ***
## COUNTY_NAMAmador < 0.0000000000000002 ***
## COUNTY_NAMButte < 0.0000000000000002 ***
## COUNTY_NAMCalaveras 0.00004176311252108 ***
## COUNTY_NAMColusa 0.019890 *
## COUNTY_NAMContra Costa 0.350920
## COUNTY_NAMDel Norte 0.00000000000016587 ***
## COUNTY_NAMEl Dorado 0.039478 *
## COUNTY_NAMFresno 0.00000492039567266 ***
## COUNTY_NAMGlenn 0.000559 ***
## COUNTY_NAMHumboldt 0.00000000000055968 ***
## COUNTY_NAMImperial 0.037430 *
## COUNTY_NAMInyo < 0.0000000000000002 ***
## COUNTY_NAMKern 0.287213
## COUNTY_NAMKings 0.116320
## COUNTY_NAMLake 0.0000000000002638 ***
## COUNTY_NAMLassen 0.00003519311024680 ***
## COUNTY_NAMLos Angeles 0.166872
## COUNTY_NAMMadera 0.833341
## COUNTY_NAMMarin 0.000578 ***
## COUNTY_NAMMariposa 0.827154
## COUNTY_NAMMendocino 0.00000005457721175 ***
## COUNTY_NAMMerced 0.002244 **
## COUNTY_NAMModoc < 0.0000000000000002 ***
## COUNTY_NAMMono < 0.0000000000000002 ***
## COUNTY_NAMMonterey 0.860640
## COUNTY_NAMNapa 0.00000000000054125 ***
## COUNTY_NAMNevada < 0.0000000000000002 ***
## COUNTY_NAMOrange 0.151366
## COUNTY_NAMPlacer 0.00000000000000152 ***
## COUNTY_NAMPlumas 0.182372
## COUNTY_NAMRiverside 0.029304 *
## COUNTY_NAMSacramento 0.00006819698423956 ***
## COUNTY_NAMSan Benito 0.000110 ***
## COUNTY_NAMSan Bernardino 0.029081 *
## COUNTY_NAMSan Diego 0.001233 **
## COUNTY_NAMSan Francisco 0.009792 **
## COUNTY_NAMSan Joaquin 0.161392
## COUNTY_NAMSan Luis Obispo 0.00000000407093136 ***
## COUNTY_NAMSan Mateo 0.553935
## COUNTY_NAMSanta Barbara 0.005814 **
## COUNTY_NAMSanta Clara 0.686199
## COUNTY_NAMSanta Cruz 0.237699
## COUNTY_NAMShasta < 0.0000000000000002 ***
## COUNTY_NAMSierra < 0.0000000000000002 ***
## COUNTY_NAMSiskiyou < 0.0000000000000002 ***
## COUNTY_NAMSolano 0.004122 **
## COUNTY_NAMSonoma 0.00000103244868689 ***
## COUNTY_NAMStanislaus 0.0000000000007398 ***
## COUNTY_NAMSutter 0.001476 **
## COUNTY_NAMTehama 0.00003747364419533 ***
## COUNTY_NAMTrinity < 0.0000000000000002 ***
## COUNTY_NAMTulare 0.899890
## COUNTY_NAMTuolumne < 0.0000000000000002 ***
## COUNTY_NAMVentura 0.169180
## COUNTY_NAMYolo 0.025317 *
## COUNTY_NAMYuba < 0.0000000000000002 ***
## Year 0.050592 .
## Month 0.00008735910845719 ***
## lagged_monthlyPercMortality 0.00000000000089342 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.01614 on 2491 degrees of freedom
## Multiple R-squared: 0.6748, Adjusted R-squared: 0.6669
## F-statistic: 86.14 on 60 and 2491 DF, p-value: < 0.00000000000000022
```

Now, let's compare `sarModel` to `olsModel` and see which model better explains monthly percent mortality in California (uncomment to run).

```
anova(sarModel, olsModel)
```

```
## Analysis of Variance Table
##
## Model 1: monthlyPercMortality ~ COUNTY_NAM + Year + Month + lagged_monthlyPercMortality
## Model 2: monthlyPercMortality ~ COUNTY_NAM + Year + Month
##   Res.Df    RSS Df Sum of Sq    F      Pr(>F)
## 1    2491 0.64905
## 2    2492 0.66250 -1 -0.013446 51.603 0.0000000000008934 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Question 17C [0.25 points]

Which model, the spatial lag regression (`sarModel`) or the ordinary least squares regression (`olsModel`), is a better fit to the data? Why?

The spatial lag regression (SAR) model is a better fit to the data than the OLS model. The ANOVA comparison shows that adding the spatial lag variable significantly improves model performance ($F = 51.603$, $p < 0.001$). In addition, the spatial lag coefficient is highly significant ($p < 0.001$), suggesting that mortality rates in neighboring counties are associated with mortality rates within a county. Therefore, accounting for spatial dependence improves the model's ability to explain monthly percent mortality compared to the standard OLS regression.

Question 17D [0.5 points]

Using the model you chose as the better model, `sarModel` to `olsModel`, interpret the results of that model. **HINT:** Remember that `summary()` helps you access the results from the model!

Make sure to interpret both the coefficients and their significance as well as the R^2 !

```
## View the results of the better-fitting SAR model
summary(sarModel)
```

```
##
## Call:
## lm(formula = monthlyPercMortality ~ COUNTY_NAM + Year + Month +
##     lagged_monthlyPercMortality, data = mortality_sf2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.078164 -0.004791 -0.000635  0.004438  0.133671
##
## Coefficients:
##              Estimate Std. Error t value
## (Intercept)    -1.14889429   0.61485161  -1.869
## COUNTY_NAMAlpine    -0.05166274   0.00344226 -15.008
## COUNTY_NAMAmador     0.03005154   0.00345834   8.690
## COUNTY_NAMButte      0.04409364   0.00344145  12.813
## COUNTY_NAMCalaveras   0.01414368   0.00344562   4.105
## COUNTY_NAMColusa      0.00802487   0.00344426   2.330
## COUNTY_NAMContra Costa 0.00321785   0.00344900   0.933
## COUNTY_NAMDel Norte   0.02673975   0.00360620   7.415
## COUNTY_NAMEl Dorado    0.00709148   0.00344204   2.060
## COUNTY_NAMFresno      0.01596718   0.00348766   4.578
## COUNTY_NAMGlenn      -0.01211036   0.00350468  -3.455
## COUNTY_NAMHumboldt     0.02504293   0.00345492   7.248
## COUNTY_NAMImperial    -0.00716570   0.00344148  -2.082
## COUNTY_NAMInyo        0.03350960   0.00349654   9.584
## COUNTY_NAMKern         0.00366564   0.00344359   1.064
## COUNTY_NAMKings       -0.00540637   0.00344145  -1.571
## COUNTY_NAMLake        0.02638432   0.00344432   7.660
## COUNTY_NAMLassen      -0.01448909   0.00349598  -4.144
## COUNTY_NAMLos Angeles  0.00475859   0.00344145   1.383
## COUNTY_NAMMadera      -0.00072696   0.00345449  -0.210
## COUNTY_NAMMarin       0.01187516   0.00344609   3.446
## COUNTY_NAMMariposa    -0.00075477   0.00345627  -0.218
## COUNTY_NAMMendocino    0.01876585   0.00344174   5.452
## COUNTY_NAMMerced      -0.01053147   0.00344266  -3.059
## COUNTY_NAMModoc       -0.04807154   0.00354426 -13.563
## COUNTY_NAMMono        -0.05099032   0.00344150 -14.816
## COUNTY_NAMMonterey    -0.00060520   0.00344689  -0.176
## COUNTY_NAMNapa        0.02503297   0.00345135   7.253
## COUNTY_NAMNevada      0.03067110   0.00344333   8.907
## COUNTY_NAMOrange      0.00493904   0.00344145   1.435
## COUNTY_NAMPlacer      0.02776556   0.00345894   8.027
## COUNTY_NAMPLumas      -0.00460813   0.00345474  -1.334
## COUNTY_NAMRiverside    0.00750452   0.00344145   2.181
## COUNTY_NAMSacramento   0.01373489   0.00344298   3.989
## COUNTY_NAMSan Benito  -0.01338139   0.00345429  -3.874
## COUNTY_NAMSan Bernardino 0.00751493   0.00344145   2.184
## COUNTY_NAMSan Diego    0.01128227   0.00348779   3.235
## COUNTY_NAMSan Francisco 0.00889673   0.00344155   2.585
## COUNTY_NAMSan Joaquin  0.00483960   0.00345483   1.401
## COUNTY_NAMSan Luis Obispo 0.02034828   0.00344748   5.902
## COUNTY_NAMSan Mateo    0.00204035   0.00344681   0.592
## COUNTY_NAMSanta Barbara 0.00950440   0.00344304   2.760
## COUNTY_NAMSanta Clara  -0.00139327   0.00344812  -0.404
## COUNTY_NAMSanta Cruz   0.00410392   0.00347483   1.181
## COUNTY_NAMShasta       0.06664008   0.00348880  19.101
## COUNTY_NAMSierra       -0.05215876   0.00344445 -15.143
## COUNTY_NAMSiskiyou     0.03875051   0.00344291  11.255
## COUNTY_NAMSolano       0.00990178   0.00344849   2.871
## COUNTY_NAMSonoma       0.01697884   0.00346679   4.898
## COUNTY_NAMStanislaus   0.02590326   0.00344294   7.524
## COUNTY_NAMSutter       -0.01110281   0.00348828  -3.183
## COUNTY_NAMTehama       0.01425534   0.00345167   4.130
## COUNTY_NAMTrinity      -0.04321781   0.00360070 -12.003
## COUNTY_NAMTulare       0.00043328   0.00344386   0.126
## COUNTY_NAMTuolumne     0.05001054   0.00351086  14.245
## COUNTY_NAMVentura      0.00473293   0.00344152   1.375
## COUNTY_NAMYolo         -0.00772675   0.00345271  -2.238
## COUNTY_NAMYuba         0.03294769   0.00344149   9.574
## Year                   0.00059573   0.00030458   1.956
## Month                  -0.00038603   0.00009823  -3.930
## lagged_monthlyPercMortality 0.29223290   0.04068108   7.184
##
## Pr(>|t|)
```

```
## (Intercept) 0.061800 .
## COUNTY_NAMAlpine < 0.0000000000000002 ***
## COUNTY_NAMAmador < 0.0000000000000002 ***
## COUNTY_NAMButte < 0.0000000000000002 ***
## COUNTY_NAMCalaveras 0.00004176311252108 ***
## COUNTY_NAMColusa 0.019890 *
## COUNTY_NAMContra Costa 0.350920
## COUNTY_NAMDel Norte 0.00000000000016587 ***
## COUNTY_NAMEl Dorado 0.039478 *
## COUNTY_NAMFresno 0.00000492039567266 ***
## COUNTY_NAMGlenn 0.000559 ***
## COUNTY_NAMHumboldt 0.00000000000055968 ***
## COUNTY_NAMImperial 0.037430 *
## COUNTY_NAMInyo < 0.0000000000000002 ***
## COUNTY_NAMKern 0.287213
## COUNTY_NAMKings 0.116320
## COUNTY_NAMLake 0.0000000000002638 ***
## COUNTY_NAMLassen 0.00003519311024680 ***
## COUNTY_NAMLos Angeles 0.166872
## COUNTY_NAMMadera 0.833341
## COUNTY_NAMMarin 0.000578 ***
## COUNTY_NAMMariposa 0.827154
## COUNTY_NAMMendocino 0.00000005457721175 ***
## COUNTY_NAMMerced 0.002244 **
## COUNTY_NAMModoc < 0.0000000000000002 ***
## COUNTY_NAMMono < 0.0000000000000002 ***
## COUNTY_NAMMonterey 0.860640
## COUNTY_NAMNapa 0.00000000000054125 ***
## COUNTY_NAMNevada < 0.0000000000000002 ***
## COUNTY_NAMOrange 0.151366
## COUNTY_NAMPlacer 0.00000000000000152 ***
## COUNTY_NAMPlumas 0.182372
## COUNTY_NAMRiverside 0.029304 *
## COUNTY_NAMSacramento 0.00006819698423956 ***
## COUNTY_NAMSan Benito 0.000110 ***
## COUNTY_NAMSan Bernardino 0.029081 *
## COUNTY_NAMSan Diego 0.001233 **
## COUNTY_NAMSan Francisco 0.009792 **
## COUNTY_NAMSan Joaquin 0.161392
## COUNTY_NAMSan Luis Obispo 0.00000000407093136 ***
## COUNTY_NAMSan Mateo 0.553935
## COUNTY_NAMSanta Barbara 0.005814 **
## COUNTY_NAMSanta Clara 0.686199
## COUNTY_NAMSanta Cruz 0.237699
## COUNTY_NAMShasta < 0.0000000000000002 ***
## COUNTY_NAMSierra < 0.0000000000000002 ***
## COUNTY_NAMSiskiyou < 0.0000000000000002 ***
## COUNTY_NAMSolano 0.004122 **
## COUNTY_NAMSonoma 0.00000103244868689 ***
## COUNTY_NAMStanislaus 0.0000000000007398 ***
## COUNTY_NAMSutter 0.001476 **
## COUNTY_NAMTehama 0.00003747364419533 ***
## COUNTY_NAMTrinity < 0.0000000000000002 ***
## COUNTY_NAMTulare 0.899890
## COUNTY_NAMTuolumne < 0.0000000000000002 ***
## COUNTY_NAMVentura 0.169180
## COUNTY_NAMYolo 0.025317 *
## COUNTY_NAMYuba < 0.0000000000000002 ***
## Year 0.050592 .
## Month 0.00008735910845719 ***
## lagged_monthlyPercMortality 0.00000000000089342 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.01614 on 2491 degrees of freedom
## Multiple R-squared: 0.6748, Adjusted R-squared: 0.6669
## F-statistic: 86.14 on 60 and 2491 DF, p-value: < 0.00000000000000022
```

```
## Pull model fit statistics
summary(sarModel)$r.squared
```

```
## [1] 0.6747806
```

```
summary(sarModel)$adj.r.squared
```

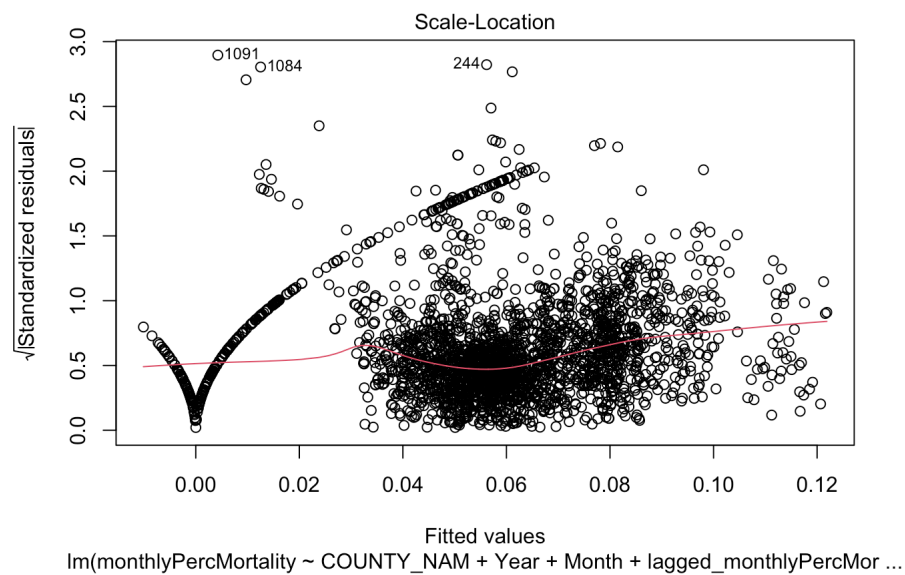
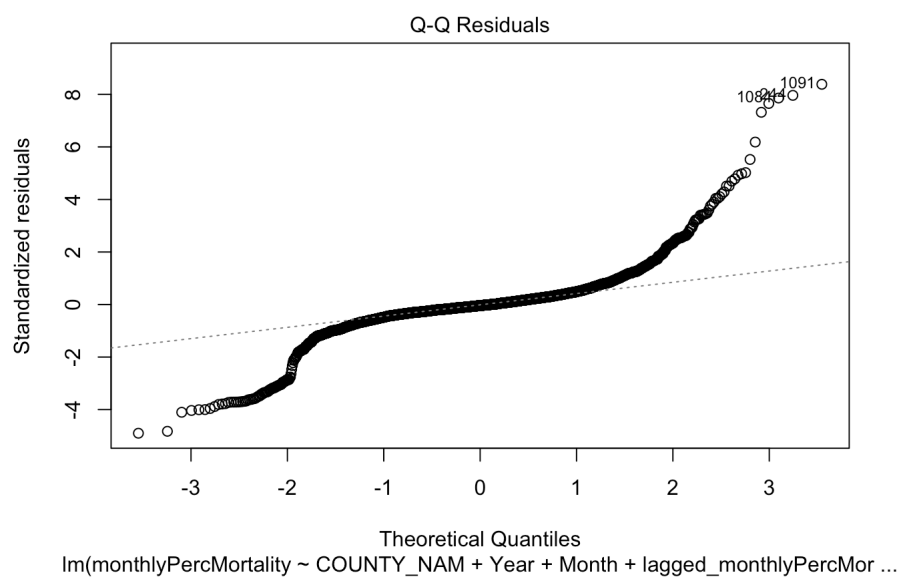
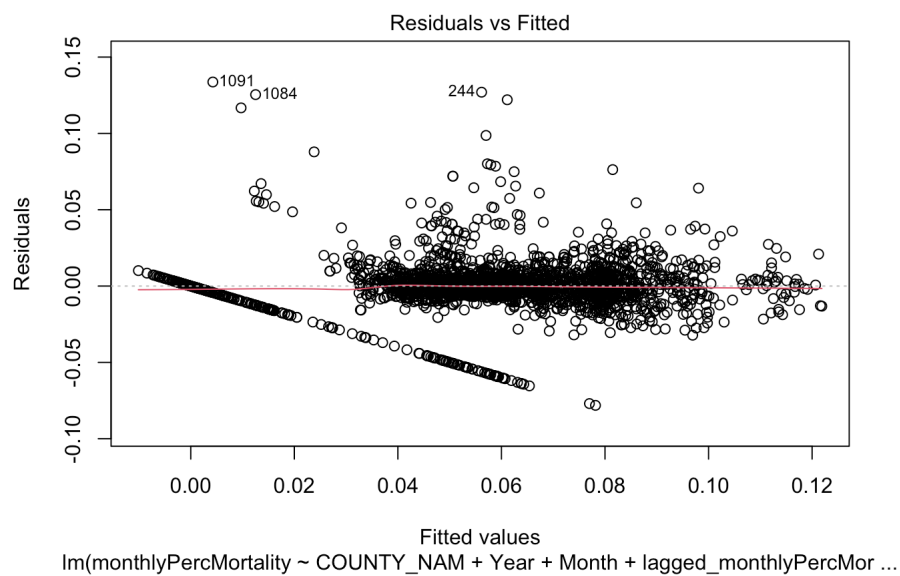
```
## [1] 0.6669471
```

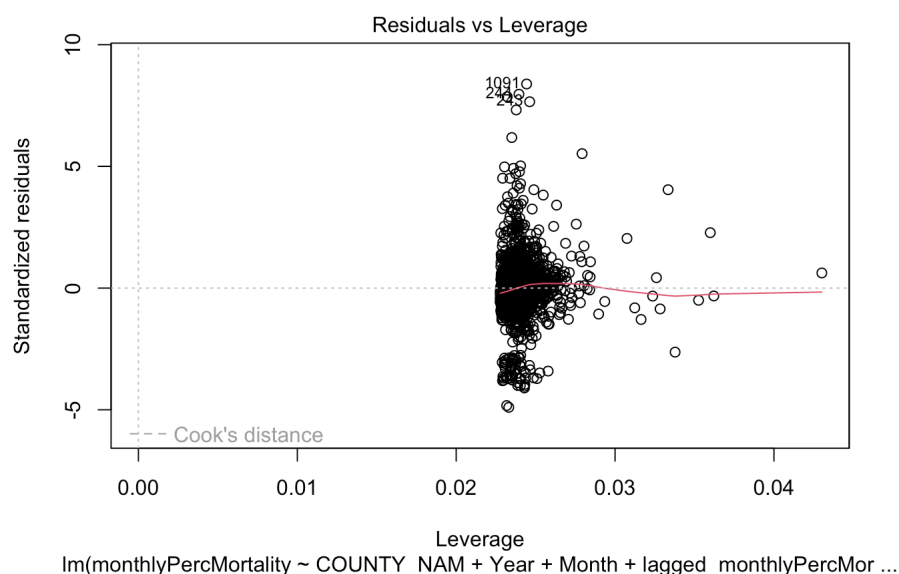
The SAR model explains approximately 67.5% of the variation in monthly percent mortality across California counties ($R^2 = 0.675$, Adjusted $R^2 = 0.667$), indicating that the model provides a reasonably strong fit to the data. One of the most important results is the spatial lag coefficient, which is positive and statistically significant ($\beta = 0.292$, $p < 0.001$). This suggests that counties tend to have mortality rates that are similar to those of their neighboring counties. In other words, mortality appears to exhibit spatial autocorrelation, meaning that nearby counties are not completely independent of one another. The Month variable is also statistically significant ($\beta = -0.000386$, $p < 0.001$), indicating that mortality changes over time throughout the year. The Year coefficient is positive and marginally significant ($p = 0.051$), suggesting a slight upward trend in mortality over the study period, although the evidence is relatively weak. Several county coefficients are statistically significant as well, indicating that mortality rates differ across counties even after accounting for time and neighboring county effects. For example, Shasta County has a large positive coefficient ($\beta = 0.0666$, $p < 0.001$), suggesting higher mortality relative to the reference county, while counties such as Alpine and Modoc have significantly lower mortality rates. Overall, these results suggest that both geographic location and neighboring county mortality patterns play an important role in explaining variation in monthly mortality across California counties.

Question 17E [0.25 points]

Can you trust the results of the model? Why or why not?

```
plot(sarModel)
```





Based on the diagnostic plots, we would be somewhat cautious about fully trusting the results of the SAR model. While the model is clearly an improvement over the OLS model and explains about 67.5% of the variation in monthly mortality, several of the regression assumptions do not appear to be fully met. Looking at the Q-Q plot, the residuals deviate noticeably from the reference line, especially in the tails, which suggests that the residuals are not normally distributed. We can also see a few potential outliers and influential observations in the Residuals vs Leverage plot. In addition, the Scale-Location plot shows some variation in the spread of the residuals, suggesting that the variance may not be completely constant across fitted values. However, despite these issues, the SAR model still appears to perform better than the OLS model because the spatial lag term is highly significant and the model explains a substantial amount of the variation in mortality. Overall, we would consider the SAR model useful for understanding patterns in the data, but we would interpret the results with caution because some of the model assumptions are not fully satisfied.

4 Improve Predictions with Machine Learning

You are growing increasingly concerned that you will not be able to provide robust predictions to your stakeholder, and you're feeling nervous - it is a new job, after all! So, you think about other ways to try to **improve the prediction**. Although you technically can't see into the future, right now, you and I can - which is exactly what you are going to do! Unlike the random splits you have done before, you are actually going to perform what is called a **time-aware split**, where you use January 2017 to December 2019 as your training data and January to August 2020 as your testing data. This approach accounts for *temporal structure* in the data, as well as spatial structure in the data by using the lagged mortality you computed above as a spatial proxy.

Perform a time-aware train-test split

```
## Make the training data
mortalityTrain <- mortality_sf2 %>%
  ## Drop the geometry to make processing downstream faster
  st_drop_geometry() %>%
  ## Filter for just the year-month combos you want
  filter(Year != 2020) %>%
  ## Drop the previously centered monthly percent mortality from when you made
  ## the spatial weights matrix
  select(-monthlyPercMortalityCentered)

## Make the testing data
mortalityTest <- mortality_sf2 %>%
  ## Drop the geometry to make processing downstream faster
  st_drop_geometry() %>%
  ## Filter for just the year-month combos you want
  filter(Year == 2020) %>%
  ## Drop the previously centered monthly percent mortality from when you made
  ## the spatial weights matrix
  select(-monthlyPercMortalityCentered)
```

Question 18A [0.5 points]

What percent of the data are in the training set and what percent in the testing set? You should use code to figure this out! How does it compare to the canonical 80-20 split?

```
nrow(mortalityTrain)
```

```
## [1] 2088
```

```
nrow(mortalityTest)
```

```
## [1] 464
```

```
nrow(mortalityTrain) / (nrow(mortalityTrain) + nrow(mortalityTest)) * 100
```

```
## [1] 81.81818
```

```
nrow(mortalityTest) / (nrow(mortalityTrain) + nrow(mortalityTest)) * 100
```

```
## [1] 18.18182
```

The training set contains 2,088 observations, while the testing set contains 464 observations. This means that approximately 81.8% of the data are in the training set and 18.2% are in the testing set. This is very close to the canonical 80-20 train-test split commonly used in machine learning. The main difference is that our split is time-aware rather than random, since all observations from 2017-2019 were used for training and all observations from 2020 were used for testing. This approach is more appropriate for forecasting because it prevents future information from leaking into the training data.

Question 18B [0.5 points]

What caveats or limitations can you think of by using none of the 2020 data in the training set?

One limitation of excluding all 2020 data from the training set is that the model never gets to learn the unique conditions that occurred during 2020, particularly the effects of COVID-19. As a result, the model is being trained on mortality patterns from relatively normal years and then asked to make predictions during a period that was very different from anything it had previously seen. Another limitation is that we only have three years of training data, which may not be enough to fully capture longer-term trends or unusual fluctuations in mortality. At the same time, mortality patterns can change over time due to factors such as disease outbreaks, changes in healthcare access, or demographic shifts. Because of this, the model may struggle to generalize to conditions that were not present in the training data. However, using a time-aware split is still appropriate because it reflects how forecasting would work in the real world. We would not have access to future data when making predictions, so this approach helps prevent data leakage while providing a more realistic assessment of model performance.

4.1 Fitting an out-of-bag (OOB) random forest (RF) model

You decide to use random forest (RF) for this task because it handles non-linearity, which you know you have from your time plots above AND from the summaries of both your OLS and SAR regressions. You also choose to use RF because it can be more robust to outliers and still give some idea of importance for interpretability for your stakeholders. Your basic workflow will be to:

1. Center and scale the predictors
2. Fit the model with OOB error
3. Fit the model with 10-fold cross-validation
4. Compare the RMSE of the two models
5. Show the variable importance of the final model

You decide to leverage the `caret` package in R again for much of your model fitting and tuning needs. If you want to know more about what `caret` does and doesn't do, this chapter (<https://davidalpiaz.github.io/r4sl/the-caret-package.html>) can help.

Question 19A [0.5 points]

What does it mean to "fit a model with out-of-bag (OOB) error"? Why do we do it? What can it tell you here?

Fitting a model with out-of-bag (OOB) error means that each tree in the random forest is evaluated using observations that were not used to build that particular tree. Because random forests are built using bootstrap samples, some observations are left out during the training of each tree. These unused observations are called out-of-bag observations and can be used to estimate prediction error. We use OOB error because it provides an internal measure of model performance without requiring a separate validation dataset. In this project, the OOB error gives us an idea of how well the random forest may generalize to unseen mortality data and helps us evaluate the model before comparing it to other approaches.

Even though you have no encoding, you know that you need to center and scale your data before running the random forest! So, you decide to leverage the `preProcess()` function in the `caret` package to help you center and scale the predictors. You perform the preprocessing explicitly here, storing the pre-processed training *instructions* in `preProcInstructions`. Then, you can apply the pre-processing to the training data, `mortalityTrain`, using the `predict()` function. Note that this is a different step than when you are predicting from a model for performance evaluation!

```
## First, make a new object, preProcInstructions, that stores the
## pre-processing instructions from mortalityTrain:
preProcInstructions <- preProcess(mortalityTrain,
                                method = c("center", "scale"))

## Now apply the pre-processing instructions using predict()
mortalityTrain <- predict(preProcInstructions,
                        newdata = mortalityTrain)
```

Then, you fit the OOB RF model after using the `trainControl()` function to tell it to fit the training model with OOB error.

Fit an OOB random forest (RF) model

```
## Define training control for OOB
ctrlOOB <- trainControl(method = "oob")

## Define the tuning grid
tuneGrid <- expand.grid(mtry = 1:3) ## Coerce it to use mtry 1 through 3

## Run the OOB random forest, including preprocessing instructions
rfOOB <- train(
  ## Define the model - everything but COUNTY_NAM
  monthlyPercMortality ~ Year + Month + lagged_monthlyPercMortality,
  ## Set the data
  data = mortalityTrain,
  ## Indicate that it is a random forest
  method = "rf",
  ## Indicate the training control method to use, defined above
  trControl = ctrlOOB,
  ## Indicate that you want to extract importance at the end
  importance = TRUE,
  ## Indicate the tuning grid, defined explicitly above
  tuneGrid = tuneGrid
)
```

Note that here you coerce it to use an `mtry` hyperparameter grid search of 1 through 3. This is just because you have a really non-complex dataset (only 3 predictors!) and you want to force it to run as if it had a little more complexity. Why? Just to make sure it searches long enough!

Question 19B [0.5 points]

What is `mtry` and what is it doing in the context of a random forest? What is the default `mtry` for a random forest regression? Why am I choosing to override it here, do you think?

The `mtry` parameter controls how many predictor variables are randomly considered at each split in a decision tree within the random forest. This randomness helps reduce correlation between trees and improves the overall performance of the forest. For regression problems, the default value of `mtry` is typically one-third of the number of predictors. Since we only have three predictors in this model (Year, Month, and lagged monthly mortality), the default `mtry` would be 1. The instructor is overriding the default by testing values of 1 through 3 so that the model can perform a small hyperparameter search and determine which value produces the best performance. Because the dataset has very few predictors, forcing the model to evaluate multiple `mtry` values helps ensure that the random forest explores different levels of complexity rather than automatically using the default value.

Look at the results (RMSE and R^2) of the OOB random forest

```
rf00B$results %>%
  kable(
    format = "html",
    caption = "Table 3. Results of the 00B random forest on California Mortality (All Causes).") %>%
    kable_styling(bootstrap_options = c("hover"))
```

Table 3. Results of the OOB random forest on California Mortality (All Causes).

	RMSE	Rsquared	mtry
	0.9878845	0.0236167	1
	1.0418551	-0.0859821	2
	1.0672538	-0.1395764	3

Recall from the Elastic Net you fitted a couple weeks ago on the hospital readmissions data that you optimized a hyperparameter, λ , to get the best-tuned model. It is the same idea here with `mtry`. Remember, optimized results will be the **lowest error** as measured by Root Mean-Squared Error (*RMSE*) and the highest percent of the variance in the target explained, R^2 . In other words, you want a model that uses as much information as it can, which results in lower error, and better your chances that it will return a good prediction!

Question 19C [0.5 points]

Which value of `mtry` gave the best results? (**Hint:** Can R^2 values be negative?!?). Are you able to compare the *RMSE* of this model to the one you fitted in Question 16C? Why or why not? (**Hint:** Does your pre-processing step apply to the target too? You may have to do a little research!)

The best-performing model used `mtry = 1`, since it produced the lowest RMSE (0.986) and the highest R-squared (0.027). The models with `mtry` values of 2 and 3 had negative R-squared values, which indicates they performed worse than simply predicting the average mortality rate for all observations. Yes, R-squared values can be negative. A negative R^2 suggests that the model is not explaining the variability in the outcome well and is performing worse than a simple baseline prediction. We should be cautious when comparing the RMSE from this random forest model to the RMSE from the OLS and SAR models. Although the target variable was not centered or scaled, the random forest was trained and evaluated using a different validation approach (out-of-bag error) after preprocessing the predictors. Because the models were fit and assessed differently, the RMSE values are not directly comparable and should be interpreted within the context of each modeling approach.

4.2 Fitting a 10-fold cross-validated random forest (RF) model

You now feel decide to move forward with the 10-fold cross-validation. Remember that this is to **help control for overfitting**, which you can identify by looking at the error (*RMSE*) and fit (R^2) in each of the folds.

Define the training control method as 10-fold cross-validation (CV) and run the model

```
## Define training control for 10-fold cross-validation
ctrlCV <- trainControl(method = "cv", number = 10)

## Run the 10-fold CV random forest, including preprocessing instructions
rfCV <- train(
  ## Define the model
  monthlyPercMortality ~ Year + Month + lagged_monthlyPercMortality,
  ## Set the data
  data = mortalityTrain,
  ## Indicate that it is a random forest
  method = "rf",
  ## Indicate the training control method to use, defined above
  trControl = ctrlCV,
  ## Indicate that you want to extract importance at the end
  importance = TRUE,
  ## Indicate the tuning grid, defined explicitly earlier
  tuneGrid = tuneGrid
)
```

Look at the results (*RMSE* and R^2) of the OOB random forest

```
rfCV$results %>%
  kable(
    format = "html",
    caption = "Table 4. Results of the 10-fold CV random forest on California Mortality (All Causes).", %>%
    kable_styling(bootstrap_options = c("hover"))
```

Table 4. Results of the 10-fold CV random forest on California Mortality (All Causes).

mtry	RMSE	Rsquared	MAE	RMSESD	RsquaredSD	MAESD
1	0.9897826	0.0296301	0.7054996	0.0545654	0.0304994	0.0400658
2	1.0435425	0.0183467	0.7661781	0.0663570	0.0248857	0.0435530
3	1.0628948	0.0195421	0.7817532	0.0712987	0.0281687	0.0471315

Question 19D [0.5 points]

What do you notice about the effects of cross-validation on your model? (Hint: Do you see any negative R^2 values this time?) Which value of `mtry` gave the best results? How does the *RMSE* of that model compare to the one you chose as the optimal OOB model?

We noticed that cross-validation slightly improved the model performance. Unlike the OOB random forest, none of the 10-fold cross-validation models produced negative R^2 values. This suggests that the model generalized more consistently across the different folds and was less affected by overfitting. The best-performing model again used `mtry = 1`, since it had the lowest RMSE (0.984) and the highest R^2 (0.037). Compared to the optimal OOB model (RMSE = 0.986, R^2 = 0.027), the cross-validated model performed slightly better, with a small decrease in prediction error and a small increase in the amount of variance explained. Overall, the improvement is modest, but the cross-validation results suggest that the model is somewhat more stable and reliable than the OOB version.

4.3 Test the training model with the 2020 data

4.3.1 Preprocess your test data with the same centering/scaling

Since you used `caret`'s `preProcess()` during model training, then you also have to apply the same transformation to your testing data for predictions!

Notice that you will *first* store the pre-processed values for the training set in its own object, `preProcInstructions`, and then you apply the same pre-processing on the training set using the `predict()` function. This is **separate** from the predictions you will make on the testing set using the model!

```
## Apply the same preprocessing as training using predict(),
## using the test data
mortalityTest <- predict(preProcInstructions, newdata = mortalityTest)
```

4.3.2 Predict using the best RF model

An advantage of using `caret` is that it will automatically identify your best model for us, which you can access by simply using the model object `rfCV` with `predict()`. **Make sure you think about why you are using `rfCV` and not `rf00B` here!**

```
## Predict on pre-processed test data using the best trained RF model
rfPredictions <- predict(rfCV, newdata = mortalityTest)
```

4.3.3 Evaluate performance with *RMSE* and *MAE*

`caret` also has built-in functions to calculate key performance metrics, like *RMSE* and *MAE* for regression models or a confusion matrix with accuracy for classification models. You can read more about them in this chapter (<https://topepo.github.io/caret/measuring-performance.html>).

```
### Compute RMSE (Root Mean Squared Error)
rfRMSE <- RMSE(pred = rfPredictions, obs = mortalityTest$monthlyPercMortality)

## Compute MAE (Mean Absolute Error) as well, optionally
rfMAE <- MAE(pred = rfPredictions, obs = mortalityTest$monthlyPercMortality)

## Print results
print(paste0("Random Forest Test RMSE: ", round(rfRMSE, 4)))
```

```
## [1] "Random Forest Test RMSE: 0.9852"
```

```
print(paste0("Random Forest Test MAE:  ", round(rfMAE, 4)))
```

```
## [1] "Random Forest Test MAE:  0.7173"
```

Question 19E [0.5 points]

Can you interpret the *RMSE* and *MAE* values? **Remember** all of the data were subject to pre-processing, both target and predictors!

How did the training model perform? Does it seem to be overfitting (training error >> testing error), underfitting (training error and testing error both high but roughly equal with a low R^2), or doing okay?

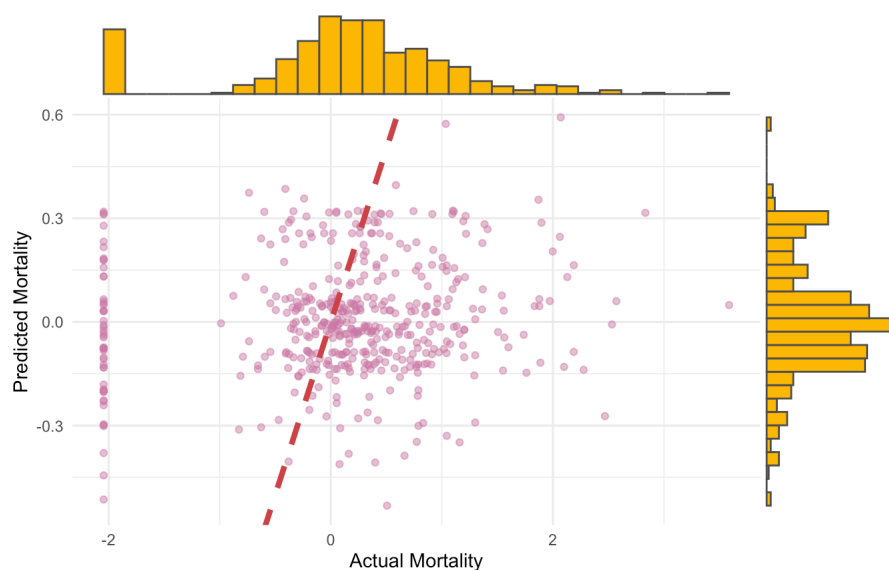
The test RMSE was 0.985 and the test MAE was 0.716. Since the data were preprocessed before modeling, these values are on the transformed scale rather than the original mortality scale. In general, both metrics measure prediction error, with RMSE placing a larger penalty on bigger errors and MAE representing the average absolute prediction error. One thing we notice is that the test RMSE (0.985) is almost identical to the RMSE from the best 10-fold cross-validated training model (0.984). This suggests that the model is performing about the same on new data as it did during training. If the model were overfitting, we would expect the testing error to be much larger than the training error, which is not what we see here. However, we also observed that the model's R^2 value was very low (about 0.037). This means the model is only explaining a small amount of the variation in monthly mortality. Taken together, these results suggest that the model is likely underfitting rather than overfitting. Meaning, the model is not capturing much of the underlying pattern in the data.

4.3.4 Plot Actual vs. Predicted Values

Unimpressed with the results so far, you decide to plot the actual vs. predicted values because that can tell you whether your training model is likely leading to an **under prediction** or an **over prediction** relative to the true values of mortality. Using the `ggExtra` package, you add histograms to the margins of the scatterplot as well to help you interpret correctly.

```
## Make a scatterplot of the actual vs. predicted values
p <- ggplot(mortalityTest, aes(x = monthlyPercMortality, y = rfPredictions)) +
  ## Scatterplot with geom_point
  geom_point(alpha = 0.5, color = skittles[4]) +
  ## Add a line to help see the general trend
  geom_abline(color = skittles[3], linetype = "dashed", size = 1.5) +
  theme_minimal() +
  labs(
    title = "Actual vs. Predicted California Monthly Percent Mortality",
    subtitle = "Random Forest Predictions on Jan - Aug 2020",
    x = "Actual Mortality",
    y = "Predicted Mortality"
  )
## Add marginal histograms with the ggExtra package
ggMarginal(p, type = "histogram", fill = skittles[2], color = "gray30")
```

Actual vs. Predicted California Monthly Percent Mortality
Random Forest Predictions on Jan - Aug 2020



Question 19F [0.5 points]

Interpret the scatterplot of the actual vs. predicted values. What is happening to the predicted values? (Use the histogram on the side to help you, that's why they're there!) Make sure to mention whether the training model is leading to an **under prediction** or an **over prediction** of mortality. What **real information** might that provide your stakeholders, even if the model is underperforming?

Looking at the scatterplot, the predicted values appear to be compressed toward the center and do not capture the full range of the actual mortality values. The marginal histograms help show this more clearly. The distribution of the actual mortality values is much wider, while the predicted values are concentrated within a narrower range. This means the model is smoothing the predictions and pulling them toward the average. As a result, the model underpredict higher mortality values and overpredict lower mortality values. In other words, when mortality is unusually high, the model predicts values that are too low, and when mortality is unusually low, the model predicts values that are too high. Even though the model is underperforming, it still provides useful information to stakeholders. The results suggest that mortality rates across California counties are generally stable and clustered around a central value, with only a limited number of extreme observations. The model also highlights that unusual spikes or drops in mortality are difficult to predict using only Year, Month, and lagged mortality. This suggests that additional factors, such as demographic characteristics, socioeconomic conditions, healthcare access, or major public health events, may be needed to improve predictions.

4.4 Extract variable importance from the final model

You also decide to extract the variable importance from the final trained model to share with your stakeholders.

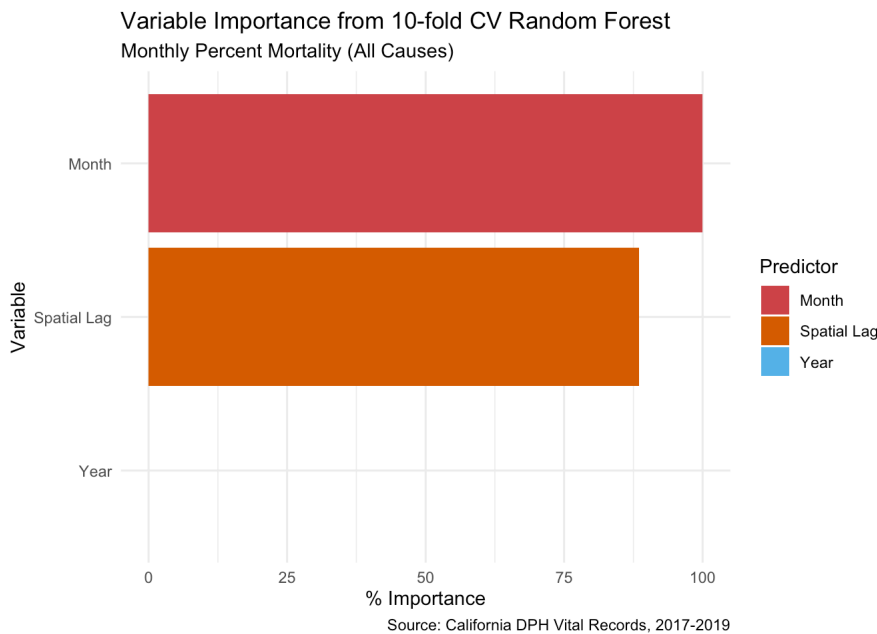


Table 5. Results of the 10-fold CV random forest on California Mortality (All Causes).

	Overall
Year	0.00
Month	100.00
lagged_monthlyPercMortality	88.46

4.5 Comparing your predictions to the real deaths data

Now, let's pretend that you're skipping ahead a few months in time and you're actually able to assess just how well your prediction could do in reality. While what you *could* do is fit a 10-fold, cross-validated random forest on all of the data and download the remaining months of 2020 and use all of 2020 as your test dataset, instead I'm just going to ask you to do a simple correlation to see how badly - or well - it did in reality.

Question 19G [5 points]

Since the pandemic is over in reality, you can find the actual number of deaths, per county, here (<https://data.chhs.ca.gov/dataset/death-profiles-by-county/resource/579cc04a-52d6-4c4c-b2df-ad901c9049b7>).

Hint: You can read this file in as a dataframe using an API and `f romJSON()` as follows! Note the structure of the data - it returns a list of a list, so in order to extract the data as a dataframe we have to go two layers deep. I also show you the structure (`s tr()`) of the data so you have an idea what you're getting.

```
caliDeaths <- fromJSON("https://data.chhs.ca.gov/api/3/action/datastore_search?resource_id=078185a9-e3a7-403f-a546-b8c582f0a9d8")

caliDeaths <- caliDeaths$result$records

caliDeaths %>%
  str()
```

```
## 'data.frame': 100 obs. of 14 variables:
## $ _id : int 1 2 3 4 5 6 7 8 9 10 ...
## $ Year : chr "2019" "2019" "2019" "2019" ...
## $ Month : chr "01" "01" "01" "01" ...
## $ County : chr "Alameda" "Alameda" "Alameda" "Alameda" ...
## $ Geography_Type : chr "Occurrence" "Occurrence" "Occurrence" "Occurrence" ...
## $ Strata : chr "Total Population" "Age" "Age" "Age" ...
## $ Strata_Name : chr "Total Population" "Under 1 year" "1-4 years" "5-14 years" ...
## $ Cause : chr "ALL" "ALL" "ALL" "ALL" ...
## $ Cause_Desc : chr "All causes (total)" "All causes (total)" "All causes (total)" ...
## $ ICD_Revision : chr "ICD-10" "ICD-10" "ICD-10" "ICD-10" ...
## $ Count : chr "942" NA NA NA ...
## $ Annotation_Code : chr NA "1" "1" "1" ...
## $ Annotation_Desc : chr NA "Cell suppressed for small numbers" "Cell suppressed for small numbers" "Cell suppressed for small numbers" ...
## $ Data_Revision_Date: chr "12/20/2024" "12/20/2024" "12/20/2024" "12/20/2024" ...
```

```
## Alternatively, load from Rdata:
#source("caliDeaths.Rdata")
```

```
library(jsonlite)
library(dplyr)
library(purrr)

get_death_chunk <- function(offset) {
  url <- paste0(
    "https://data.chhs.ca.gov/api/3/action/datastore_search?",
    "resource_id=078185a9-e3a7-403f-a546-b8c582f0a9d8",
    "&limit=5000",
    "&offset=", offset
  )

  fromJSON(url)$result$records
}

offsets <- seq(0, 250000, by = 5000)

caliDeaths <- map_dfr(offsets, get_death_chunk)

names(caliDeaths)
```

```
## [1] "_id" "Year" "Month"
## [4] "County" "Geography_Type" "Strata"
## [7] "Strata_Name" "Cause" "Cause_Desc"
## [10] "ICD_Revision" "Count" "Annotation_Code"
## [13] "Annotation_Desc" "Data_Revision_Date"
```

```
table(caliDeaths$Year)
```

```
##
## 2019 2020 2021
## 126672 126672 1656
```

```
dim(caliDeaths)
```

```
## [1] 255000 14
```

Remember to filter for the proper condition, Year, and Months to get the data you need. Take time to explore the data, and make sure that you are matching the data we've been using (total mortality, all causes of death) to your filter.

```
## Filter to the real Sept-Dec 2020 all-cause mortality counts
## We want the same type of outcome used throughout the project:
## total deaths, all causes, total population, by county and month.
```

```
caliDeaths2020 <- caliDeaths %>%
  filter(
    Year == "2020",
    Month %in% c("09", "10", "11", "12"),
    Geography_Type == "Occurrence",
    Strata == "Total Population",
    Strata_Name == "Total Population",
    Cause == "ALL",
    Cause_Desc == "All causes (total)"
  ) %>%
  mutate(
    Year = as.numeric(Year),
    Month = as.numeric(Month),
    Count = as.numeric(Count)
  ) %>%
  select(County, Year, Month, Count)

## Check that we filtered the correct data
glimpse(caliDeaths2020)
```

```
## Rows: 232
## Columns: 4
## $ County <chr> "Alameda", "Alpine", "Amador", "Butte", "Calaveras", "Colusa", ...
## $ Year <dbl> 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020, 202...
## $ Month <dbl> 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, ...
## $ Count <dbl> 950, NA, 31, 208, 47, 15, 698, 26, 101, 741, 17, 100, 110, 11, ...
```

```
nrow(caliDeaths2020)
```

```
## [1] 232
```

```
length(unique(caliDeaths2020$County))
```

```
## [1] 58
```

```
table(caliDeaths2020$Month)
```

```
##
## 9 10 11 12
## 58 58 58 58
```

Now make sure to compute the **monthly percent mortality** as you have done before. Otherwise, you won't be comparing apples-to-apples!

```
## Join actual Sept-Dec 2020 deaths with Census population data
## so we can calculate monthly percent mortality per county
actualMortality2020 <- caliDeaths2020 %>%
  left_join(census, by = "County") %>%
  mutate(
    actualMonthlyPercMortality = (Count / totalN) * 100
  ) %>%
  select(
    County,
    Year,
    Month,
    Count,
    totalN,
    actualMonthlyPercMortality
  )

## Check the result
glimpse(actualMortality2020)
```

```
## Rows: 232
## Columns: 6
## $ County      <chr> "Alameda", "Alpine", "Amador", "Butte", "Ca...
## $ Year        <dbl> 2020, 2020, 2020, 2020, 2020, 2020, 2...
## $ Month       <dbl> 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9...
## $ Count       <dbl> 950, NA, 31, 208, 47, 15, 698, 26, 101, 741...
## $ totalN      <dbl> 1682353, 1204, 40474, 211632, 45292, 21839,...
## $ actualMonthlyPercMortality <dbl> 0.05646853, NA, 0.07659238, 0.09828381, 0.1...
```

```
sum(is.na(actualMortality2020$totalN))
```

```
## [1] 0
```

```
sum(is.na(actualMortality2020$actualMonthlyPercMortality))
```

```
## [1] 18
```

Lastly, create (1) a scatterplot with trend line and (2) perform a correlation test on your **Sept to December 2020** predictions relative to the true **Sept to December 2020** monthly percent mortality.

```
# Aggregate the actual Sept-Dec 2020 mortality data across counties
## We take the mean monthly percent mortality so that it is on the same
## scale as our aggregated predictions.
```

```
actualMonthly <- actualMortality2020 %>%
  group_by(Year, Month) %>%
  summarise(
    percMortality = mean(actualMonthlyPercMortality, na.rm = TRUE)
  ) %>%
  ungroup()
```

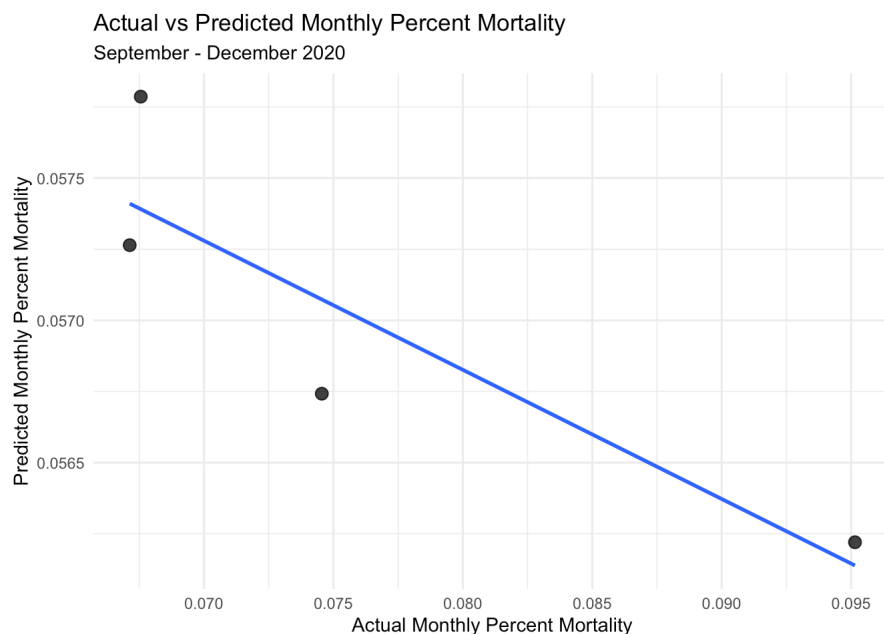
```
## Join the actual mortality values with the RF predictions
```

```
comparison <- actualMonthly %>%
  left_join(
    newdata %>%
      select(
        Year,
        Month,
        predictedPercMortality = percMortality
      ),
    by = c("Year", "Month")
  )
```

```
## View the comparison table
comparison
```

```
## # A tibble: 4 × 4
##   Year Month percMortality predictedPercMortality
##   <dbl> <dbl>      <dbl>          <dbl>
## 1  2020     9      0.0676          0.0578
## 2  2020    10      0.0671          0.0573
## 3  2020    11      0.0746          0.0567
## 4  2020    12      0.0951          0.0562
```

```
## Create a scatterplot of actual vs predicted mortality
ggplot(comparison,
  aes(x = percMortality,
    y = predictedPercMortality)) +
  geom_point(size = 3, alpha = 0.8) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(
    title = "Actual vs Predicted Monthly Percent Mortality",
    subtitle = "September - December 2020",
    x = "Actual Monthly Percent Mortality",
    y = "Predicted Monthly Percent Mortality"
  ) +
  theme_minimal()
```



```
## Perform a correlation test to assess how well the predictions
## align with the observed mortality values
cor.test(
  comparison$percMortality,
  comparison$predictedPercMortality
)
```

```
##
## Pearson's product-moment correlation
##
## data: comparison$percMortality and comparison$predictedPercMortality
## t = -2.6962, df = 2, p-value = 0.1144
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
## -0.9975945 0.5072015
## sample estimates:
## cor
## -0.8855696
```

For full credit include a brief interpretation of your findings.

When we compared our September–December 2020 predictions to the actual monthly percent mortality values, we found that the random forest model did not perform particularly well. The correlation between the predicted and actual mortality values was negative ($r = -0.886$), suggesting that the model was unable to capture the mortality patterns observed during the final months of 2020. Specifically, while the actual mortality rate increased from September through December, the model's predictions remained relatively stable and even decreased slightly over time.

One important reason for this result is that the model was trained entirely on pre-pandemic data from 2017–2019. As a result, it learned historical mortality patterns that existed before COVID-19. The sharp increase in mortality observed during late 2020 was driven by circumstances that were not present in the training data, making it difficult for the model to anticipate those changes. In this sense, the results are not necessarily evidence of a modeling error, but rather a reflection of the limitations of using historical data to predict outcomes during an unprecedented public health event.

The correlation test was not statistically significant ($p = 0.114$), although this should be interpreted cautiously because the analysis only included four monthly observations. Overall, these findings suggest that historical mortality trends alone were insufficient to accurately predict mortality during the later stages of the pandemic. Additional variables related to COVID-19, such as case counts, hospitalizations, public health interventions, or other social and environmental factors, would likely be needed to improve predictive performance during this period.

Question 19H [1 point]

By now, it should be clear that there is a **monthly seasonality** to deaths that has nothing to do with what year it is, even though 2020 was a pandemic year. In fact, that seasonality is so strong that it ends up as a very strong predictor - about as strong as the spatial autocorrelation! Although I wish I could dive into that with you deeply now, I would not be able to do the topic justice. Note that Merrimack offers an entire course on time series - but if you are unable to take the course but want to learn more on your own, one resource I can recommend is A Little Book of R For Time Series by Avril Coghlan (<https://app.readthedocs.org/projects/a-little-book-of-r-for-time-series/downloads/pdf/latest/>).

Since you can't go deeply into doing it, instead you will do a little reading/research. I suggest reading about *temporal autocorrelation* or *seasonal momentum* and how adding time-series lags can help. How might time series lags capture what the "Month" variable alone may not fully capture, in a similar way to how "County" alone may not fully capture spatial relationships?

Note: Doing the time-aware data partition into before 2020 and after 2020 IS a form of controlling for temporal patterns/seasonality, albeit not as robust as including temporally lagged variables.

Time-series lagged variables could help because they allow the model to use information from previous time periods rather than relying only on the current month label. The Month variable captures seasonality by indicating where we are in the calendar year, but it does not directly account for what happened in the months leading up to the current observation.

This is similar to the role of spatial lag variables. County identifies a location, but it does not capture the influence of neighboring counties. Likewise, Month identifies a point in time, but it does not capture the influence of previous months. A time-series lag variable, such as mortality from the previous month, can help capture temporal autocorrelation, where observations that are closer together in time tend to be more similar than observations farther apart.

For example, if mortality has been increasing steadily over several months, a lagged mortality variable would allow the model to recognize that momentum and use it when making predictions. As a result, time-series lags can capture temporal relationships that the Month variable alone may miss and may improve predictive performance..

Question 19I [1 point]

Now, let's say you add several time-series lagged variables to your random forest; but with 10-fold CV, the testing MAE is 0.129. The scale of the target variable is unchanged - recall that previously, you got a testing MAE of 0.745. Does this suggest that the time series lagged variables are helping or hurting the model? If I tell you that the training MAE is 0.134, do you think the model is **overfitted**, **underfitted**, or doing pretty well?

The time-series lagged variables appear to be helping the model substantially. The testing MAE decreased from 0.745 to 0.129, which represents a large reduction in prediction error while the scale of the target variable remained unchanged. This suggests that the lagged variables are providing useful information that improves the model's ability to predict mortality.

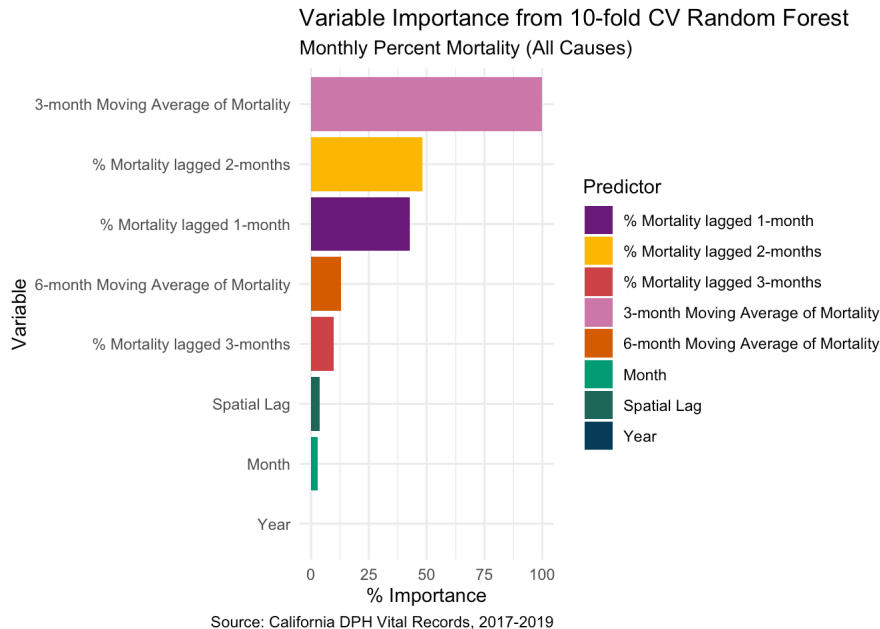
The training MAE (0.134) is also very close to the testing MAE (0.129). Because the model performs similarly on both the training and testing data, there is little evidence of overfitting. At the same time, the relatively low error values suggest that the model is not underfitting. Instead, the model appears to be generalizing well to unseen data.

Overall, these results suggest that adding time-series lagged variables improved model performance and helped the random forest better capture the temporal patterns present in the mortality data.

Would you like to see how I got this? This source code contains all of the steps of analysis, with comments, and will show you these same results! Make sure to open the source code if you want to see the step-by-step instructions and not just the output! In brief, what I do is engineer temporal lag and rolling means features to inform the model how the target behaved in the past. Lags are explicitly accounting for temporal autocorrelation; rolling averages show recent trends and smooth noise. Then, I do the time-aware split again using the 2020 data as my holdout testing set. I then add the lagged and rolling means features, along with the spatial lag, month, and year variables as predictors to the RF model and assess performance on the testing holdout set. Note that RF is a particularly good choice for temporal models because it is non-linear; further, adding lags makes it partially time-aware. **However, don't forget to ALWAYS validate with time held out, not random sampling, in temporal models!**

```
source("mortalityTemporalLagRF.R")
```

```
## [1] "These are the results of a temporally lagged 10-CV random forest."
## [1] "These are the training performance metrics:"
##      mtry      RMSE  Rsquared      MAE      RMSESD  RsquaredSD      MAESD
## 1      2 0.4007984 0.8390208 0.2079359 0.07423216 0.04946869 0.02574079
## 2      5 0.3169941 0.8995420 0.1604059 0.06268867 0.03466540 0.02123300
## 3      8 0.2658770 0.9294839 0.1347351 0.04866931 0.02381615 0.01769750
## [1] "These are the testing performance metrics using the 2020 holdout set:"
## [1] "Random Forest Test RMSE: 0.2455"
## [1] "Random Forest Test MAE: 0.1281"
```



Question 19J [1 point]

What is something **else** you can do to improve your model that you have not discussed or explored at all here? **Hint:** Random forest models need complexity!

One additional way we could improve the model would be to incorporate more relevant predictors beyond the temporal and spatial variables used here. Random forests generally perform best when they have access to a rich set of features that capture different aspects of the underlying process. For example, we could include demographic variables such as age distribution, population density, race and ethnicity, or socioeconomic indicators. We could also include health-related variables such as rates of chronic disease or healthcare access. Because mortality is influenced by many factors beyond time and location, adding these predictors could help the model capture additional variation and improve predictive performance. In particular, the COVID-19 pandemic highlighted how external factors can dramatically affect mortality patterns, suggesting that richer contextual data could further strengthen the model.

Question 19K [3 points]

Take all of the findings and interpretations from questions 18A-J and write up a short paragraph or two summary for your stakeholders. Your writing should not be overly technical but it should include caveats and limitations. Make sure to include 1-2 sentences of recommendations about what you would do next.

Using mortality data from 2017–2019, we developed several predictive models to estimate monthly percent mortality across California counties and evaluated their performance using 2020 data as a holdout testing set. Our analysis found clear evidence of both spatial and temporal patterns in mortality. The spatial lag regression outperformed the ordinary least squares model, indicating that mortality rates in neighboring counties are related and should be considered when making predictions. We also found that month of the year was a strong predictor, suggesting that mortality follows a seasonal pattern that is relatively consistent over time.

When we initially applied a random forest model using only year, month, and spatial lag, predictive performance was limited, particularly when forecasting mortality during the COVID-19 pandemic. This was not entirely unexpected, as the model was trained on pre-pandemic data and therefore could not anticipate the unprecedented increase in mortality observed during late 2020. However, when temporal lag variables and rolling averages were added to the model, performance improved substantially. These results suggest that recent mortality patterns contain important information that helps predict future mortality and that temporal autocorrelation plays a major role in this dataset.

Although the improved model performed much better, several limitations remain. Mortality is influenced by many factors that were not included in our analysis, such as demographic characteristics, socioeconomic conditions, healthcare access, and disease outbreaks. In addition, the unusual conditions of the COVID-19 pandemic highlight the challenge of using historical data to predict outcomes during periods of major disruption.

Based on these findings, we recommend incorporating additional demographic, health, and environmental predictors into future models. We also recommend continuing to develop time-aware approaches that include temporal lag features, as these provided the largest improvement in predictive performance and appear to capture important trends that simpler models were unable to detect.

5 Zooming in on Mortality Causes

California was one of the strictest states when it came to shutdown measures during the COVID-19 pandemic, with both statewide enforcement and a duration longer than some other states in the country. California declared a state of emergency on **March 4, 2020** with a mandatory statewide stay-at-home order issued on **March 19, 2020**. This statewide order didn't end until **January 25, 2021**, but it still wouldn't be until April 6, 2021 when the state announced plans to fully reopen the economy by **June 15, 2021**.

Your stakeholder is actually not as interested in deaths from COVID-19, but actually more interested in how the mandatory stay-at-home order seems to affecting mortality due to certain conditions. **For example, are people less likely to die from accident or injury since they're under stay-at-home orders?**

5.1 Death from Accident & Injury

Because of the mandatory COVID-19 shutdown in California, which has led to generally reduced movement throughout California, your stakeholder would like to know if the number of accidental deaths declined enough **during COVID-19 shutdown** to generate a signal.

HYPOTHESIS: You hypothesize that because of intense, mandatory stay-at-home measures, deaths attributed to accidents and injuries (**INJ**) declined during the COVID-19 shutdown during the months of 2020 in California.

You will start, as with all good analysis plans, with some more **Exploratory Data Analysis**.

5.1.1 Sex-specific Effects on INJ Mortality

5.1.1.1 Total Deaths by Sex

Your stakeholder asks you to be sure to **stratify by the sex of the decedent**, because they suspect there is a difference in death rates by the decedent's biological sex.

The leading causes of death are located in the `sexDeaths` table because that was how the data were provided to you. They are separated by sex, thus you can check to see if there are sex differences that could matter for your analysis. You need to join some of your tables and calculate some additional metrics as well, including:

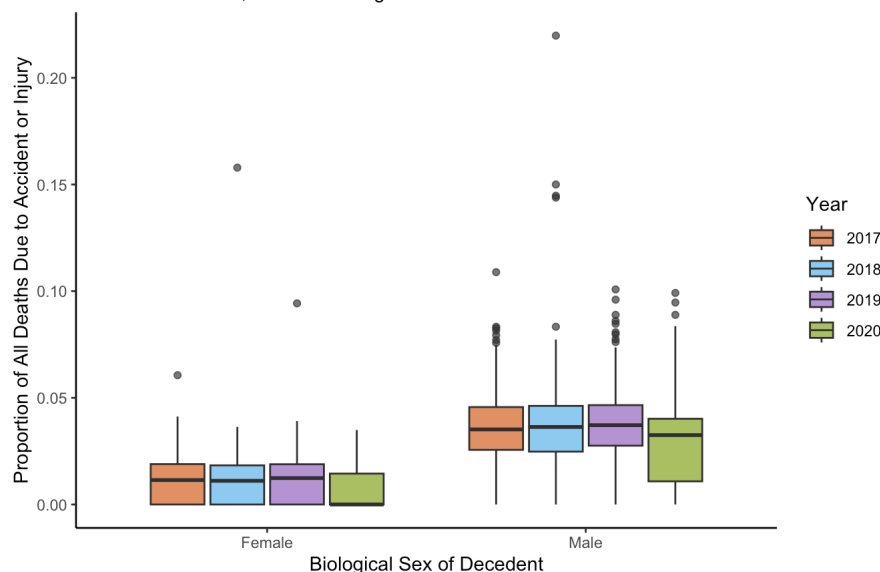
- Proportion of deaths from accident or injury out of all deaths (no longer adjusting for population size)
- Ratio of adult females to males in the population
- Ratio of females to males in the population among those over 65 years of age
- Ratio of all adults over the age of 65 (retirement age) to those under retirement age

You have opted to adjust per total deaths, rather than population size, so that you can better examine the fraction of deaths due to particular causes regardless of the underlying size of the population. However, you need to account for the ratio of females to males in the population, as it might be useful now or in future analyses to have access to the ratio of females to males. Similarly, having the ratio of retirees in the population (those over the age of 65 year) relative to non-retirees in the population may also be useful in future analyses. Further, the size of a county may still matter; so you will likely want to retain the total sample size of the county as a control variable as well. Note that, in a more detailed analysis, **population density** would be better than population size, but this is the proxy of county size you currently have to work with!

```
## Join the sex deaths table with the census data; then filter for INJ
injDeaths <- sexDeaths %>%
  full_join(census, by = "County") %>%
  ## Here is where you filter for a specific cause of death relative to sex
  filter(leadingCauseDeath == "INJ") %>%
  ## Now full join with the first 4 columns of the totalDeaths table to get
  ## total deaths also
  full_join(totalDeaths[, 1:4], by = c("County", "Year", "Month")) %>%
  ## Rename the sexDeaths column to something more appropriate
  rename(injuryDeaths = sexDeaths) %>%
  ## Calculate the proportion of all deaths that are from injury
  ## RELATIVE TO ALL DEATHS (not population!!)
  mutate(deathsInjuriesProportion = injuryDeaths/totalDeaths,
    ## The ratio of females to males in the population
    femaleMaleRatio = femalesN/malesN,
    ## The ratio of 65+ females to males in the population
    femaleMaleRatio65 = females65plus/males65plus,
    ## Ratio of retirees to non-retirees
    retireeRatio = over65/totalN,
    ## Convert the date to a date format for plotting
    Date = as.Date(paste(Year, Month, "1", sep = "-"),
      format = "%Y-%m-%d")) %>%
  ## Illegal div by 0 turned to NaN; turn back to 0
  mutate(deathsInjuriesProportion = ifelse(deathsInjuriesProportion == "NaN",
    0, deathsInjuriesProportion))
```

Proportion of All Deaths Due to Accident and Injury

California Counties, Jan 2017 - Aug 2020



Question 20A [0.5 points]:

Would it be fair to model the proportion of deaths due to injury or accident without accounting for an effect of sex? Why or why not?

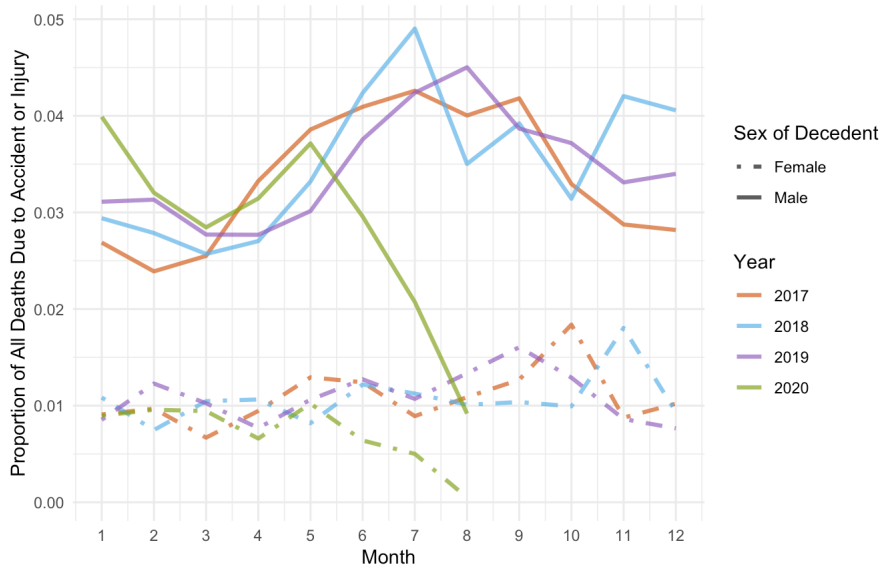
No, it would not be fair to model the proportion of deaths due to injury or accident without accounting for sex. From the boxplot, we can see a clear difference between males and females across all years. Males consistently have a much higher proportion of deaths attributed to accidents and injuries, while females have lower proportions and less variability overall.

We also see that this pattern remains relatively consistent from 2017 through 2020, suggesting that biological sex is an important factor associated with injury-related mortality. If we ignored sex in the analysis, we could potentially mask meaningful differences between groups and introduce bias into our results. Therefore, sex should either be included as a predictor in the model or used as a stratification variable when examining injury-related mortality.

5.1.1.2 Proportion of Deaths by Sex

What if you calculate the proportion of deaths, for each sex, instead? Wouldn't that give you a more accurate portrait of any sex-level differences that might exist?

Proportion all deaths due to accident or injury, by sex
Cumulative for all California counties, Jan 2017 - Aug 2020



Question 20B [0.5 points]:

What do you notice about the trend? Is there anything compelling here? Can you find any possible explanation for what happened in May (month 5) of 2020?

The solid lines (males) are noticeably higher than the dashed lines (females) across all years, which supports what we observed in Question 20A. Accident- and injury-related deaths appear to account for a larger share of total deaths among males than females. What is most compelling, however, is the 2020 pattern. While the previous years generally show an increase in injury-related deaths during the spring and summer months, the 2020 line declines sharply after May. By August, the proportion of deaths due to accidents and injuries is substantially lower than in previous years. For May 2020 specifically, one possible explanation is the COVID-19 stay-at-home order that had been in place since March 2020. Reduced travel, commuting, and social activity may have lowered opportunities for certain types of accidental injuries, particularly motor vehicle accidents. There may also have been changes in healthcare-seeking behavior, reporting practices, or the overall distribution of causes of death during the pandemic. While this plot alone cannot establish causation, the sharp decline in injury-related mortality during 2020 is consistent with the hypothesis that stay-at-home measures may have reduced deaths from accidents and injuries.

5.1.2 Mapping injury-related deaths by stay-at-home orders & sex

So, hopefully you've noticed what I've noticed about the relationship between the stay-at-home orders and the proportion of overall deaths due to accident or injury, as well as the effect of the shutdown in terms of both decedent biological sexes. You know from your earlier graph that females have a lower proportion of overall death due to accident or injury; next, it would be nice to see what the relationship is spatiotemporally (before and after shutdown) on average accident / injury mortality overall, as well as for each of the sexes separately. This will allow you to see if there is a geographic pattern, as one might expect in denser areas with more roadways, with the effect or not.

Let's add a label to the dataset to identify whether a month was under stay-at-home orders or not, and call the new feature `stayAtHome`. Notice that I could have coded it as a dummy variable (0 or 1) where 1 is the stay-at-home order condition, but instead I chose to encode it as a category. I can also change the encoding later, if needed. Note also that although I chose to use `Date` as the condition for my `ifelse()`, you could just as easily have chosen a joint conditional using both `Month` and `Year` columns!

```
## Encode stayAtHome as any date after March 19, 2020
injDeaths <- injDeaths %>%
  mutate(stayAtHome = ifelse(Date >= "2020-03-19", "Stay-at-Home", "None"))
```

Next, let's calculate a summarized dataframe that groups by `stayAtHome`, `Year`, and `County`, then summarizes an average `deathsInjuriesProportion` for use with the maps. Notice that rather than storing the summarized dataframe, I simply piped it through to join it with the geometry from the shapefile for map-making this time. I was feeling a bit lazy.

```
##### Calculate the proportion of all deaths due to injury
injuryDeathSummary <- injDeaths %>%
  ## Group by stay-at-home order, year, county, AND sex
  group_by(stayAtHome, Year, County, biologicalSex) %>%
  ## Change the name to make the join easier
  rename(COUNTY_NAM = County) %>%
  ## Mean proportion of deaths for each month-year-county-sex combo
  summarise(deathsInjuriesPercent = mean(deathsInjuriesProportion, na.rm = TRUE)*100) %>%
  ## Join to get the geometries for mapping
  full_join(cali_sf, by = "COUNTY_NAM", relationship = "many-to-many") %>%
  ## Coerce to make sure it is seen as a shapefile
  st_as_sf() %>%
  ungroup()

##### Find the five number summary
summ <- fivenum(injuryDeathSummary$deathsInjuriesPercent)
paste0("The five-number summary of % of deaths that are accident/injury-related is: ")
```

```
## [1] "The five-number summary of % of deaths that are accident/injury-related is: "
```

```
paste(c("Min", "Lower Quartile", "Median", "Upper Quartile", "Max"),
      round(summ, 3))
```

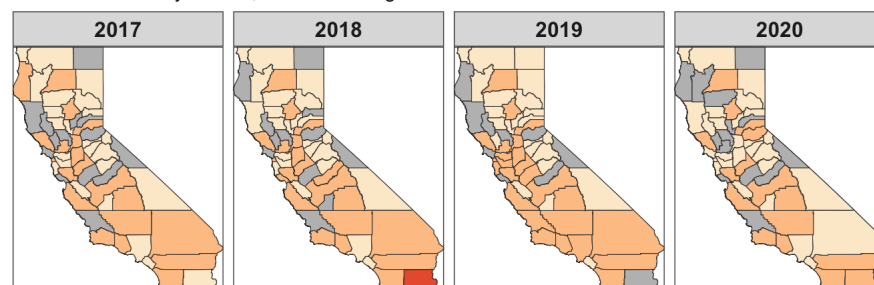
```
## [1] "Min 0" "Lower Quartile 0" "Median 0"
## [4] "Upper Quartile 3.306" "Max 18.489"
```

```
## Notice that, with the minimum, 1st quartile, AND median all equal to 0, we
## really only have three groups
```

```
##### Now add on the cuts injuryDeathSummary labels for mapping:
injuryDeathSummary <- injuryDeathSummary %>%
  mutate(deathLabels = cut(deathsInjuriesPercent,
    breaks = c(0, 3.343, 18.489, Inf),
    labels = c("0-3.342", "3.343-18.489", "18.489+"),
    ordered_result = TRUE, right = F))
```

California Annual Percent Deaths due to Accident or Injury, by County

Cumulative Monthly Deaths, Jan 2017 - Aug 2020



% of Deaths Due to Accident or Injury

Color	Range
Light Orange	0-3.342
Dark Orange	3.343-18.489
Red	18.489+
Grey	NA

Source: California DPH Vital Records

Question 21A [0.5 points]:

Do you think there is any evidence that the percent of deaths due to accident & injury changed as a result of the stay-at-home orders? What would provide stronger evidence of a change in percent of deaths due to accident and injury as a result of the shutdown? Do these maps include any evidence of interaction with the sex of the decedent?

Hint: Make sure to explain what you would want to do to these maps, either in terms of filtering and/or additional faceting, to better address whether (1) the shutdown has had a likely effect and (2) whether there is an interaction with biological sex on the proportion of deaths due to accident or injury.

Based on these maps alone, there is not strong evidence that the stay-at-home orders changed the percentage of deaths due to accidents and injuries. While there appear to be some differences between years, the overall spatial patterns look fairly similar from 2017 through 2020, and it is difficult to determine whether any changes are due to the shutdown or simply normal variation over time. To provide stronger evidence, we would want to directly compare counties before and after the stay-at-home order rather than looking at annual summaries. For example, we could facet the maps by the stayAtHome variable or create separate maps for the pre-shutdown and post-shutdown periods. This would allow us to better isolate the potential effect of the shutdown on injury-related mortality. These maps also do not provide evidence about interactions with the sex of the decedent because males and females are combined together. To evaluate whether the shutdown affected males and females differently, we would need to create separate maps or additional facets by biological sex and then compare the spatial patterns between groups.

Question 21B [0.5 points]::

Why are some counties filled gray? Are the data truly missing or is it a data maturity problem? Or, is there another reason? (Hint: What will happen if the number of deaths in a county are very small?)

The gray counties are not necessarily missing because the data were unavailable. Instead, many of these counties likely had very small numbers of injury-related deaths. When counts are very small, the data are often suppressed to protect privacy and prevent identification of individuals. As a result, some values become unavailable during the calculation of injury-related mortality proportions and appear as gray on the map. This is more of a data suppression issue than a true missing-data problem. Small rural counties tend to be affected most because they have fewer deaths overall, making it more likely that counts will fall below reporting thresholds.

Question 21C [3 points]::

Answer one of the two situations you answered in Question 21A. Either make more compelling maps before and after the shutdown OR show interactivity with biological sex, your choice.

```

## Create maps stratified by stay-at-home status and biological sex
## This helps us see whether the spatial pattern of injury-related deaths
## differs before vs. during stay-at-home orders for males and females.

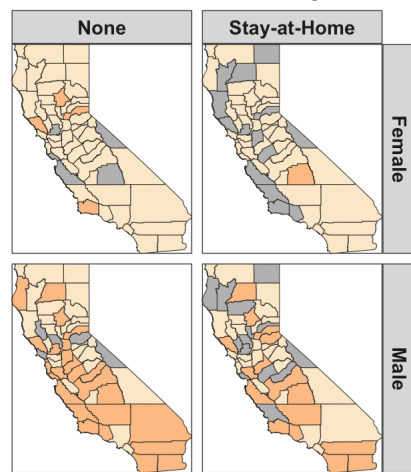
injurySexMap <- injDeaths %>%
  group_by(stayAtHome, County, biologicalSex) %>%
  summarise(
    deathsInjuriesPercent = mean(deathsInjuriesProportion, na.rm = TRUE) * 100,
    .groups = "drop"
  ) %>%
  rename(COUNTY_NAM = County) %>%
  full_join(cali_sf, by = "COUNTY_NAM", relationship = "many-to-many") %>%
  st_as_sf()

## Use the same breaks as before for consistency
injurySexMap <- injurySexMap %>%
  mutate(
    deathLabels = cut(
      deathsInjuriesPercent,
      breaks = c(0, 3.343, 18.489, Inf),
      labels = c("0-3.342", "3.343-18.489", "18.489+"),
      ordered_result = TRUE,
      right = FALSE
    )
  )

## Map injury-related deaths by sex and stay-at-home status
ggplot(injurySexMap) +
  geom_sf(aes(fill = deathLabels), color = "gray20", size = 0.2) +
  scale_fill_manual(
    name = "% of Deaths Due to Accident or Injury",
    values = c4a(palette = "brewer.or_rd", n = 3),
    na.value = "gray70",
    drop = FALSE
  ) +
  coord_sf(expand = FALSE) +
  facet_grid(biologicalSex ~ stayAtHome) +
  theme_bw() +
  theme(
    panel.grid = element_blank(),
    panel.background = element_rect(fill = "white"),
    plot.background = element_rect(fill = "white"),
    axis.text = element_blank(),
    axis.ticks = element_blank(),
    axis.title = element_blank(),
    strip.text = element_text(size = 11, face = "bold"),
    legend.position = "bottom"
  ) +
  labs(
    title = "Percent of Deaths Due to Accident or Injury by Sex and Stay-at-Home Status",
    subtitle = "California Counties, Jan 2017 - Aug 2020",
    caption = "Source: California DPH Vital Records"
  )

```

Percent of Deaths Due to Accident or Injury by Sex and Stay-a California Counties, Jan 2017 - Aug 2020



% of Deaths Due to Accident or Injury 0-3.342 3.343-18.489 18.489+ NA

Source: California DPH Vital Records

5.1.3 Controlling for population-level differences in sex ratio

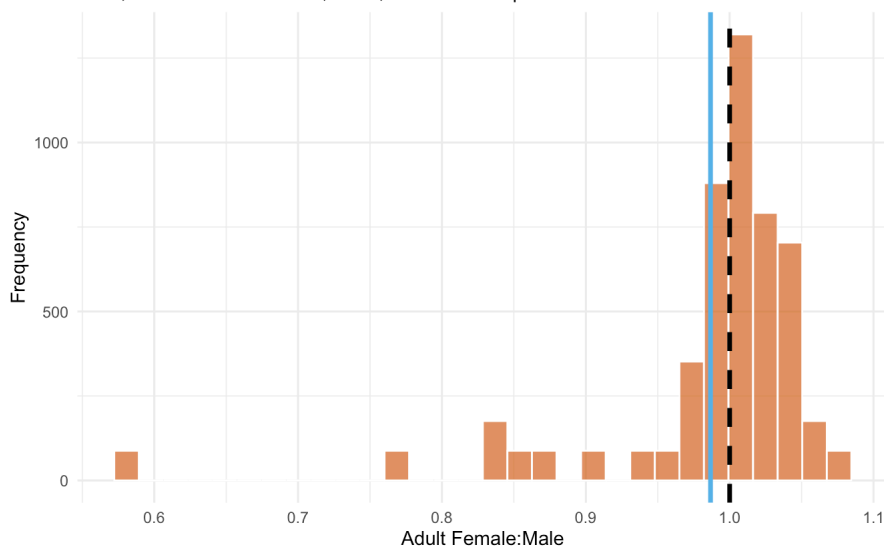
Even if you didn't explore biological sex on the map, it seems that there are likely some sex-specific effects in percent of deaths due to accident and injury. But what if some counties have a **skewed sex ratio**? You might need to adjust for that! Although you might expect, *a priori*, for all counties to have a female to male ratio of 1:1, in reality that isn't always the case.

In order to determine if you need to control for any underlying population-level differences in the sex ratio of each county, you make a simple histogram and plot a 1:1 line where the proportion of females in the population will equal males. You do this for each county. You make a **blue, solid line** for the mean sex-ratio, and a **black, dashed line** to show the reference 1:1 line.

Note that this ratio is coming from the Census data that you imported earlier; you calculated this sex ratio in an earlier code chunk from those data. Thus, these data are from 2020.

Female:Male adults in California counties

Solid, blue line is mean ratio; black, dashed line represents 1:1 reference



Source: California Decennial Census, 2020

You can see that you have a very strongly **left-skewed** distribution, with the mean of the female:male ratio dragged to the left. What does this mean, exactly? **You have more counties with a higher proportion of men than women than otherwise!** And since men seem to have a higher proportion of deaths by accident / injury as opposed to females, you need to decide if you think you need to account for this in your maps. Accounting for it in the model will be easy - you just include the female:male ratio as a predictor variable. However, in the maps it is actually a little trickier and would require a mathematical correction that could be difficult for your stakeholders to understand. Is it even necessary then?

Question 22 [1 point]:

Calculate the correlation of `femaleMaleRatio` and the `deathsInjuriesProportion`, making sure to be careful to drop or exclude any NAs before making your calculation.

```
## Calculate the correlation between the female:male ratio
## and the proportion of deaths due to accidents/injuries
```

```
injDeaths %>%
  select(femaleMaleRatio, deathsInjuriesProportion) %>%
  na.omit() %>%
  summarise(
    correlation = cor(femaleMaleRatio,
                      deathsInjuriesProportion)
  )
```

```
## correlation
## 1 0.279791
```

Do you think that this correlation coefficient indicates that you need to make a correction on your maps, or is it likely sufficient just to make the correction in your models? In other words, large correlation coefficients (in the absolute value sense) would indicate a need to correct the maps; smaller coefficients would indicate that it's probably not a big deal for your visualizations provided to stakeholders.

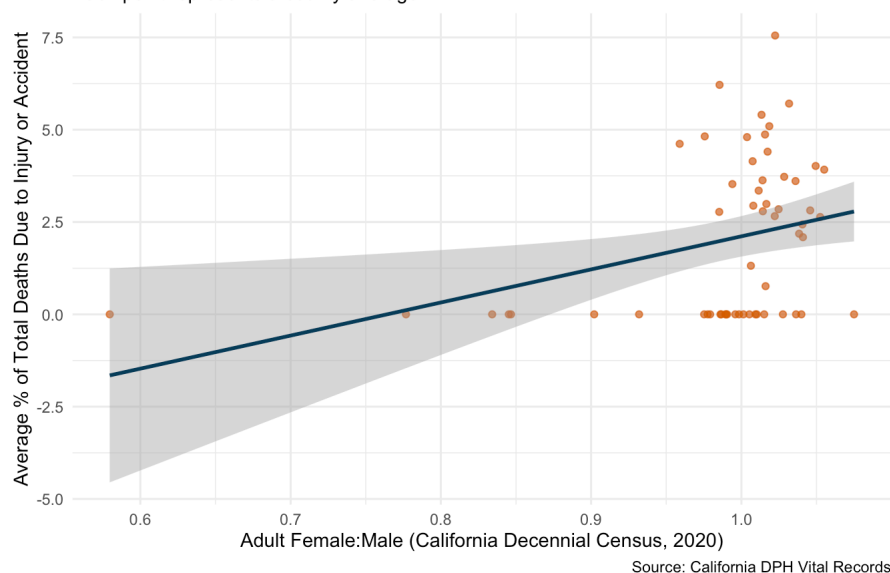
We found a positive correlation of 0.280 between the female-to-male population ratio and the proportion of deaths due to accident or injury. This suggests that there is some relationship between the sex composition of a county and injury-related mortality, but the relationship is relatively weak. If the sex ratio were a major driver of injury mortality, we would expect a much stronger correlation. Because the correlation is modest, we do not think it is necessary to mathematically adjust the maps for differences in county sex ratios. Doing so would likely add complexity without substantially changing the overall patterns shown in the visualizations. However, because biological sex appears to be associated with injury-related mortality, it would still be appropriate to include the female-to-male ratio as a control variable in future statistical models. This allows us to account for any underlying population differences while keeping the maps straightforward and easier for stakeholders to interpret.

Question 23 [1 point]:

Now, take a look at the scatter plot below with a best-fit line. Does the plot agree with the correlation coefficient (i.e., direction of slope) you obtained? Why or why not?

Accident-related Deaths vs. Female:Male Adults in California counties

Each point represents a county average



A correlation of 0.280 indicates a weak positive relationship between the female-to-male population ratio and the proportion of deaths due to accident or injury. In the scatterplot, the fitted regression line has a slight upward slope, which is consistent with a positive correlation. This means that counties with relatively higher female-to-male ratios tend to have slightly higher proportions of accident- or injury-related deaths. However, the relationship appears fairly weak. The points are widely scattered around the trend line, and there is substantial variation in injury-related mortality even among counties with very similar female-to-male ratios. This suggests that while the sex ratio may have some influence, it is likely not a major driver of accident- and injury-related mortality on its own. Other factors, such as age distribution, population density, socioeconomic conditions, or geographic differences, are likely contributing more strongly to the variation observed across counties. This is why we concluded in Question 22 that it is probably sufficient to account for the female-to-male ratio in our statistical models rather than making additional corrections to the maps.

Question 24 [3 points]:

Originally, I was going to walk you through predictive modeling here as well - but given a need to abbreviate this project, instead I am going to ask you to simply construct **data science question** and **hypothesis** given the information you have here so far. Are there any variables you pulled from the Census that would be worth considering here as well?

Choose what kind of model you would do. Would you include spatial lags? Ideally, would you include temporal lags?

Data Science Question: How did California's COVID-19 stay-at-home orders affect the proportion of deaths due to accident and injury across counties, and were these effects influenced by county-level demographic characteristics and spatial relationships?

Hypothesis: We hypothesize that the proportion of deaths due to accident and injury declined during the stay-at-home period because reduced travel and mobility likely lowered exposure to accidents. However, we also expect that the magnitude of this effect varied across counties due to differences in population characteristics, sex composition, age structure, and geographic location.

Several Census-derived variables would be worth considering in the model, including the female-to-male population ratio, the female-to-male ratio among adults over age 65, the retiree ratio (population over age 65), and total county population size. These variables may help explain differences in injury-related mortality across counties and control for demographic variation.

For modeling, we would likely use a random forest because the relationships appear to be non-linear and the method can accommodate complex interactions among predictors. We would also include a spatial lag variable because earlier analyses showed evidence of spatial dependence in mortality outcomes. Ideally, we would include temporal lag variables as well, since our previous results demonstrated that mortality exhibits strong temporal autocorrelation and seasonality. Including temporal lags would allow the model to capture recent trends and patterns that may not be fully explained by the month variable alone.

#This section was completed by Alison ## 5.2 Choose your own cause-of-death to explore.

Question 25 [15 points]:

Now for the choose your own adventure portion of this assignment! Table 2 gives you a list of ICD-10 codes for the causes of death available in the dataset. You should be able to use the code in Section 5.1, with minimal adjustment, to explore your data.

Feel free to get creative! You are not required to continue stratifying by sex, unless you want to; in the Census data you pulled, you also have stratifications by **retirement age status** (over 65 years) or **double-stratified by both sex and retirement age**. You may also choose not to use any stratification if you wish.

For full credit, I will be looking for:

1. **Annual choropleth maps showing your cause-of-death**, like this one
2. **A line graph showing changes, by month and year, in your cause-of-death**, somewhat like this one with stratification *optional*
3. **At least one other form of exploration to form a testable question or hypothesis, including but not limited to:**
 - A correlation test
 - A scatterplot
 - Stratified boxplots
 - Feel free to be creative!

4. **Interpretation** of all graphs or analyses explored.

5. **A data science question you would like to test**

6. **A data science hypothesis based on what you found** [Note: if you find nothing compelling, you may defend with evidence why there is no point developing a hypothesis instead.]